

基于动态搜索的“板凳龙”运动状态及路线研究

摘要

中药材在加工后需要及时脱除水分，以提高储藏稳定性并保持成品品质，利用热空气脱除水分是生产中常用的处理方式。本文以圆柱形药材为对象，将烘房实测温湿度作为动态边界，依次建立固定域、变物性耦合及收缩移动边界模型，求解预热与恒温干燥阶段的温度场和干基含水率场，并确定全域含水率达标所需时间。

针对问题一，将细长圆柱简化为一维轴对称介质，依据 Fourier 定律、Fick 定律和控制体守恒建立非稳态传热传质方程及表面对流边界，采用**守恒型径向有限体积法、后向 Euler 全隐式格式和 Picard 迭代**求解。1800 s 时中心与表面温度分别为 33.5753 °C 和 36.7856 °C，水分浓度分别为 2.5500 kg/kg 和 1.5102 kg/kg，表明表面响应显著快于内部。

针对问题二，引入随含水率变化的密度、比热容和导热系数，以及同时受温度和含水率影响的扩散系数，构建**变物性热湿双向耦合模型**；采用表层加密的两区非均匀网格和**自适应变阶隐式 BDF 方法**求解刚性半离散系统。3 h 时中心与表面温度为 49.8495 °C 和 49.9664 °C，水分浓度为 1.7662 kg/kg 和 1.0081 kg/kg，说明温度已接近平衡而水分梯度仍然显著。

针对问题三，沿用固定半径变物性模型，将观测区间后的边界连续过渡至稳定环境，并引入全域最大含水率构造烘干判据。利用**表层加密有限体积-隐式 BDF 框架**追踪中心阈值，求得连续临界时刻为 **57.4723 h**，含水率场综合经验误差为 9.53×10^{-7} kg/kg。空间-时间独立加密、**Chebyshev 谱元-Radau IIA 异构复算及 Bessel 解析解检验**共同验证了结果的数值稳定性。

针对问题四，将实测半径作为移动边界，建立**均匀仿射收缩-材料坐标模型**，以 $\xi = r/R(t)$ 将时变区域映射至固定区间，并结合附录 4 物性关系求解温湿场。所得烘干临界时刻为 **51.0906 h**，此时半径约为 1.2000 cm，较问题三缩短 6.3817 h；该差异同时包含收缩与物性变化。进一步利用干质量一致性诊断及两个守恒对照模型，揭示了实测几何、密度解释与干质量守恒间的结构不确定性。

本文以统一守恒框架贯通固定域与移动域计算，并通过**表面加密、刚性隐式积分、事件定位、独立收敛与跨算法复算**构成数值证据链，同时区分离散误差和模型结构误差，为药材烘干过程预测与工艺评价提供了可解释工具。

关键字： 热风烘干 热湿耦合 有限体积法 移动边界 数值验证

一、问题重述

1.1 问题背景

中药材在加工后需要及时脱除水分，以提高储藏稳定性并保持成品品质。热风方式利用空气与药材之间的温差和湿度差完成热量、水分交换，药材内部经历由外向内的升温过程，同时水分由内部逐步迁移至表面并逸出。整个过程可分为升温过渡阶段和后续恒温脱水阶段，两个阶段的传热传质特征并不完全相同。

烘房温度、湿度及物性参数的组合会直接影响干燥速率、能耗和最终品质，单纯依靠反复试验难以系统考察这些因素的作用。为此，本文从连续介质传热与传质角度描述药材内部状态的时空变化，并结合数值计算比较不同干燥阶段及尺寸变化对结果的影响，为干燥时间判定和工艺参数分析提供依据。

1.2 问题提出

如图 2 所示，研究对象取为一段长度 25 cm、半径 2 cm 的圆柱状药材。计算开始时，药材内部温度和干基含水率分别为 28 °C 和 2.55 kg/kg。附件同时提供烘房环境状态的时间序列以及干燥过程中药材半径的变化信息。本文据此建立模型并完成下列任务。

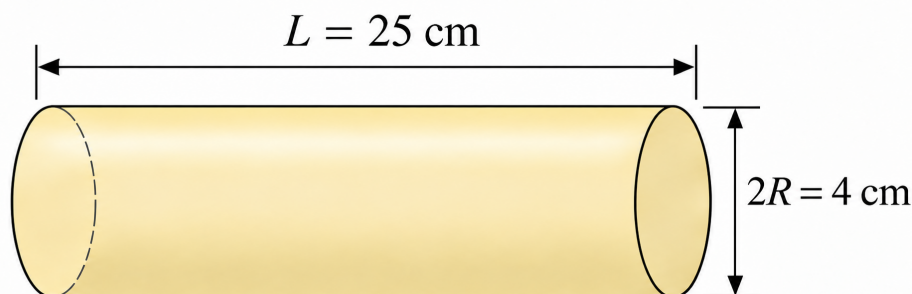


图 1 药材几何外形示意图

问题一：首先研究干燥开始后的 1800 s 预热阶段。利用附件 1 的烘房环境数据和附录 2 给出的物性参数，求解药材内部温度和含水率随时间及径向位置的分布。

问题二：在完整干燥过程的计算中，采用附录 3 的统一经验关系描述物性参数随状态的变化，建立温度场与含水率场的耦合模型。

问题三：在半径保持不变的条件下，以药材各处含水率均低于 0.15 kg/kg 作为干燥完成判据，确定达到该判据所需的时间。

问题四：进一步考虑药材失水后半径随时间收缩的情况，利用附件 2 的半径数据和附录 4 的经验关系修正传质计算域，重新确定达到 0.15 kg/kg 判据的干燥时长。

二、问题分析

2.1 问题一

问题一要求刻画预热平衡阶段药材内部温度和水分浓度随时间及径向位置的变化规律。由于药材可近似为均质、各向同性的细长圆柱体，其长度与半径之比为 12.5，两个端面的总面积仅为侧面积的 8%，因此可忽略轴向梯度和端部效应，将问题简化为固定半径圆柱内的一维轴对称非稳态传热与传质问题 (如图 3)，从而将温度场与水分场作为两个相互并行的偏微分方程分别求解。对于温度场，根据 Fourier 导热定律和能量守恒建立圆柱坐标下的一维径向非稳态热传导方程，在圆柱中心设置对称边界，在药材表面设置 Newton 对流换热边界。对于水分场，根据 Fick 扩散定律和质量守恒建立径向扩散方程。为求解上述温度场和水分场的偏微分方程，本文采用径向有限体积法进行空间离散，并采用后向 Euler 全隐式格式进行时间离散与推进。

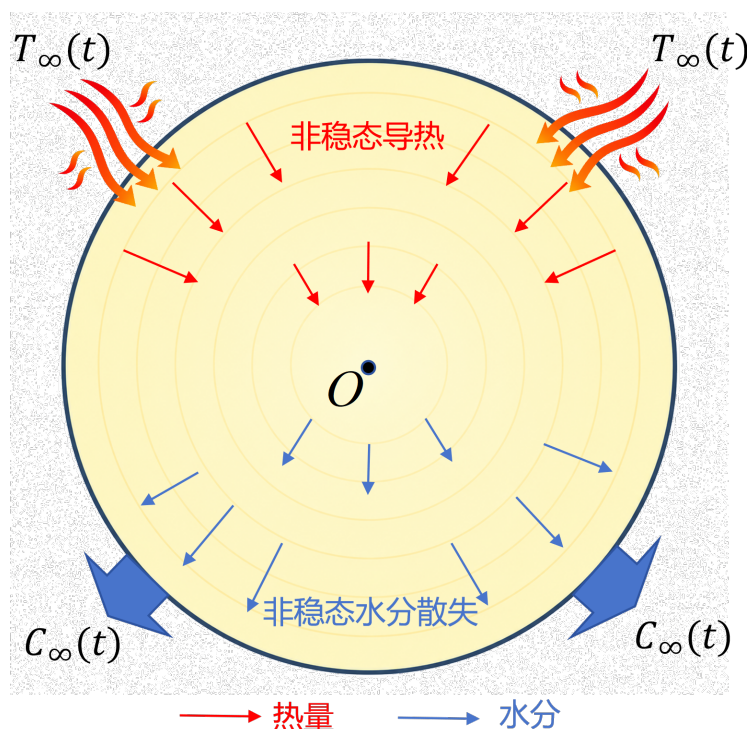


图 2 药材径向传热传质示意图

2.2 问题二

问题二沿用问题一建立的一维径向“传热-传质”模型，但进一步考虑药材物性参数随温度和含水率变化的影响。如图3所示，密度、比热容和导热系数由含水率决定，水分扩散系数则同时受含水率和绝对温度影响，因此温度场与水分场通过变物性参数形成间接双向耦合。模型从烘干初始状态开始计算，环境温度和水分浓度边界沿用分段线性插值结果。求解时采用与问题一一致的径向有限体积法和后向 Euler 全隐式格式，在每个

时间层内交替更新变物性参数并采用 Picard 迭代求解，直至温度场和水分场同时收敛，进而获得前 3 h 内各时刻、各径向位置处的温度 and 水分浓度。

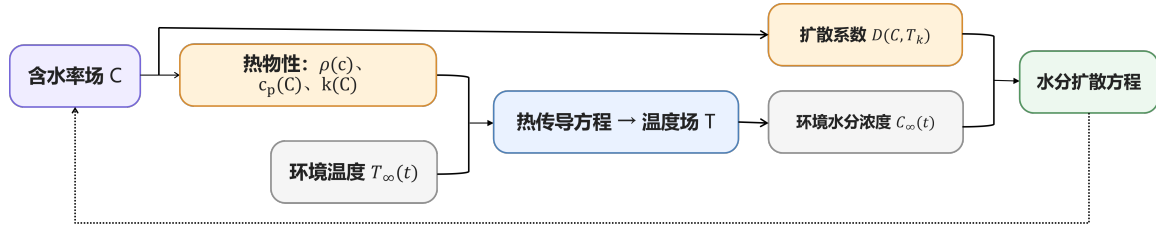


图3 问题二热湿耦合计算流程图

2.3 问题三

问题三要求确定药材各处干基含水率均低于 0.15 kg/kg 的时刻。本文继承问题二的固定半径变物性模型，先对附件 1 测量区间以外的环境边界作连续延拓，再以全域最大含水率构造终止事件。主计算采用表面加密的有限体积网格和变步长 BDF2 格式，通过独立的空间、时间加密估计数值不确定性；随后利用采用不同空间离散和时间推进的 Chebyshev 谱元-Radau 方法复算同一径向模型，检验临界时刻是否依赖单一数值实现。

2.4 问题四

问题四首先只考虑题目要求的收缩效应：将附件 2 的实测半径作为移动边界，假设圆柱长度不变且径向作均匀仿射收缩，在固定材料坐标中求解附录 4 物性下的热湿耦合方程，得到烘干临界时长。得到主结果后，再以干基含水率和真实湿密度的定义检验干固体质量，判断主模型的几何、密度与守恒关系是否相容。针对检验中发现的矛盾，进一步建立“实测半径-反演长度”和“固定长度-预测半径”两个非仿射守恒模型，分别考察放松长度约束或放松实测半径约束后的结果及代价。由此形成“主模型计算-守恒检验-对照修正-模型评价”的递进分析。

2.5 问题五

问题五的目的是建立一个优化模型，求解最大速度。首先需要在问题四的基础上，计算全过程中各把手的最大速度。其次需要对问题四的结果进行分析，结合物理规律，缩小搜索范围。最后依据速度最大值函数的特征，采用适当的算法求解最大速度。

三、建模假设

1. **连续、均质、各向同性**：在宏观尺度上，将药材等效为连续、均质且各向同性的多孔介质，忽略纹理方向、孔隙结构及分层物性等信息以及各向异性的导热和扩散参数。

2. **一维径向近似**：药材长度远大于半径，端面相对于侧面的面积较小，因此忽略周向变化和主体区域的轴向端部效应，只考虑径向传热与传质。
3. **忽略蒸发散热**：水分蒸发会带走汽化潜热，但题目未给出汽化潜热、蒸发速率及相变动力学参数，因此温度方程中不显式加入蒸发吸热项；由此产生的影响作为模型误差处理。同时忽略体热源、化学反应热、辐射换热和热扩散引起的直接水分通量。
4. **局部宏观平均**：将温度场 $T(r, t)$ 和干基含水率场 $C(r, t)$ 视为连续场变量，微观孔隙内的气液界面不单独解析，其影响等效包含在 k 和 D 中。
5. **第四问主模型的收缩方式**：主模型假设圆柱长度保持 L_0 不变，同一初始相对半径处的材料层始终具有相同的径向缩放比，即作均匀仿射收缩。该假设仅用于先行求得题设驱动的工程基线，随后将由干质量守恒检验其适用性。

四、符号说明

表 1 符号说明

符号	含义	SI 单位
L	药材长度	m
R_0	药材初始半径	m
r	到圆柱中心轴的物理距离	m
$R(t)$	当前药材半径；前三问为 R_0	m
t	时间	s
$T(r, t)$	药材温度	°C
$T_K(r, t)$	绝对温度， $T_K = T + 273.15$	K
$C(r, t)$	干基含水率，即水质量与绝干物质量之比	kg/kg
$T_\infty(t)$	烘房环境温度	°C
$C_\infty(t)$	与表面传质模型匹配的环境等效含水率	kg/kg
ρ	表观密度	kg/m ³
c_p	比热容	J/(kg · K)
k	导热系数	W/(m · K)
D	有效水分扩散系数	m ² /s
h	表面对流换热系数	W/(m ² · K)
h_m	表面对流传质系数	m/s
ξ	第四问的无量纲材料坐标， $\xi = r/R(t)$	1

药材初始长度和半径分别为 $L = 25 \text{ cm}$ 、 $R_0 = 2 \text{ cm}$ ，初始状态为 $T(r, 0) = 28^\circ\text{C}$ 、 $C(r, 0) = 2.55 \text{ kg/kg}$ 。其中 C 为干基含水率，数值 2.55 表示每 1 kg 绝干药材含 2.55 kg 水。

五、模型建立与求解

5.1 问题一模型的建立与求解

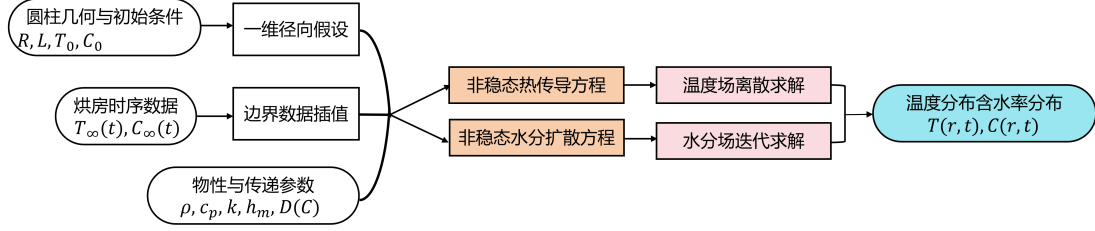


图 4 问题一流程图

5.1.1 线性插值处理

附件 1 给出的烘房温度和水分浓度为时间间隔 60 s 的离散采样序列，而数值离散的时间层间隔为 1 s。为获得各计算时间层对应的边界状态，本文对该采样序列进行分段线性插值。设相邻采样时刻为 t_j 和 t_{j+1} ，则当 $t \in [t_j, t_{j+1}]$ 时，有

$$\begin{cases} T_{\infty}(t) = T_{\infty,j} + \frac{t - t_j}{t_{j+1} - t_j} (T_{\infty,j+1} - T_{\infty,j}) \\ C_{\infty}(t) = C_{\infty,j} + \frac{t - t_j}{t_{j+1} - t_j} (C_{\infty,j+1} - C_{\infty,j}) \end{cases} \quad (1)$$

该插值仅用于构造连续变化的边界条件，不改变附件 1 的原始采样数据。

5.1.2 温度场模型

为清晰表征药材内部传热、传质过程及其边界交换关系，图5示意了一维径向非稳态热传导与水分扩散过程。模型中，药材中心取轴对称条件作为内边界，药材表面与烘房环境之间分别以对流换热和对流传质进行热量与水分交换；后续温度场与水分场控制方程即基于此建立。

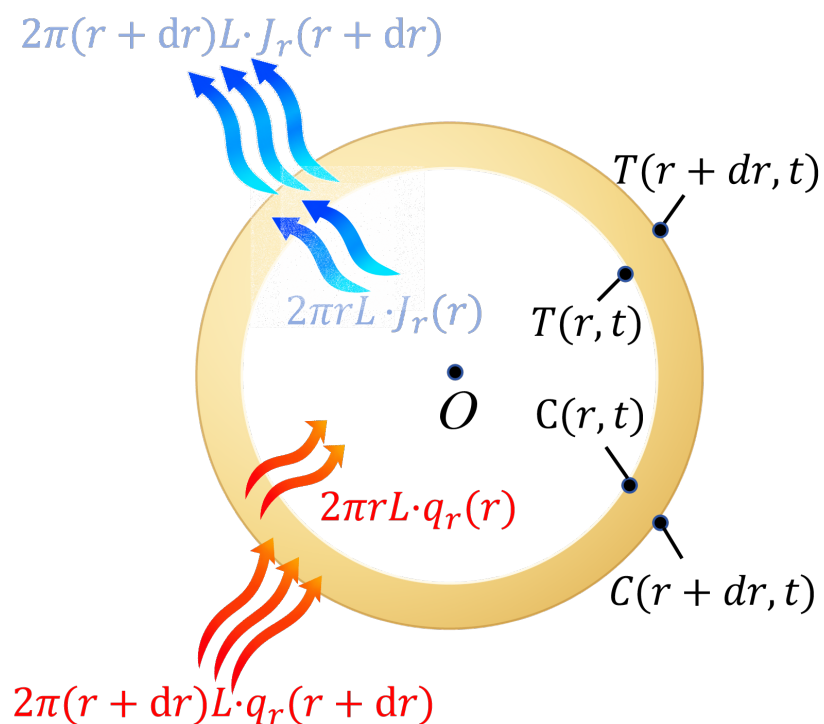


图5 一维径向非稳态热传导和水分扩散示意图

根据 Fourier 导热定律，药材内部的径向热流密度 q_r 为 $-k \frac{\partial T}{\partial r}$ 。如图6所示，取半径为 r 、厚度为 dr 、长度为 L 的圆柱薄壳作为控制体，其体积 dV 为 $2\pi r L dr$ 。

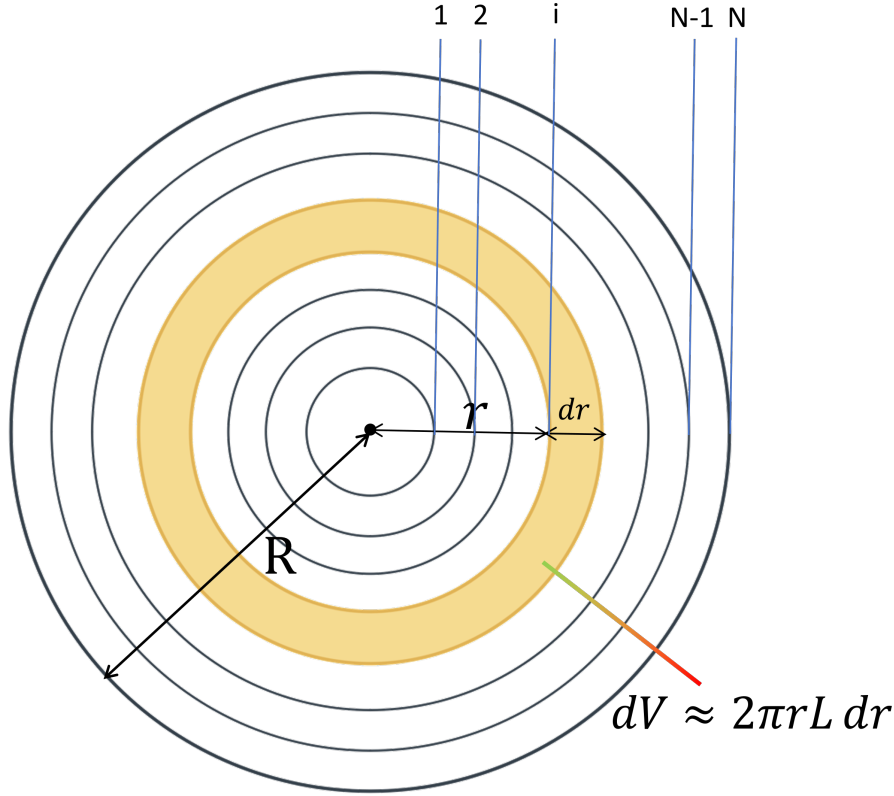


图6 圆柱薄壳控制体示意图

薄壳单位时间内的显热积累量为 $\rho c_p dV \frac{\partial T}{\partial t}$ 。薄壳的净导热输入量等于内侧流入与外侧流出之差，由径向热流密度 q_r 可写为 $-2\pi L \frac{\partial(rq_r)}{\partial r} dr$ 。根据能量守恒，显热积累量等于净导热输入量，约去公共因子 $2\pi L dr$ ，得到圆柱坐标下的一维径向非稳态热传导方程：

$$\rho c_p \frac{\partial T}{\partial t} = \frac{1}{r} \frac{\partial}{\partial r} \left(r k \frac{\partial T}{\partial r} \right) \quad 0 < r < R, \quad 0 < t \leq 1800 \text{ s}. \quad (2)$$

第一问中 ρ 、 c_p 和 k 均为常数，因此上式可写为：

$$\frac{\partial T}{\partial t} = \alpha \left(\frac{\partial^2 T}{\partial r^2} + \frac{1}{r} \frac{\partial T}{\partial r} \right), \quad \alpha = \frac{k}{\rho c_p}. \quad (3)$$

代入题目给定的物性参数，得到热扩散率 $\alpha = 1.6886 \times 10^{-7} \text{ m}^2/\text{s}$ 。

初始时刻，药材温度均匀分布，即 $T(r, 0) = 28^\circ\text{C}$ 。由于圆柱中心满足轴对称条件，有 $\frac{\partial T}{\partial r} \Big|_{r=0} = 0$ 。令 $T_s = T(R, t)$ ，药材表面向外的导热通量 q_{out} 为 $-k \frac{\partial T}{\partial r} \Big|_{r=R}$ 。根据 Newton 对流换热定律，环境向表面传出的热流为 $h(T_\infty - T_s)$ 。在药材与烘房气体的分界面上，根据能量守恒原理，药材内部导热通量与界面对流换热通量相等，得到表面对流换热边界条件：

$$-k \frac{\partial T}{\partial r} \Big|_{r=R} = h [T_s - T_\infty(t)]. \quad (4)$$

5.1.3 水分场模型

根据 Fick 第一定律 [1], 药材内部的径向水分扩散通量 J_r 为 $-D(C)\frac{\partial C}{\partial r}$ 。对同一圆柱薄壳控制体, 含水率的积累量为 $2\pi r L \mathrm{d}r \frac{\partial C}{\partial t}$, 水分的净扩散输入量为 $-2\pi L \frac{\partial(rJ_r)}{\partial r} \mathrm{d}r$ 。根据质量守恒定律, 约去公共因子并代入 J_r , 得到含水率的一维径向扩散方程

$$\frac{\partial C}{\partial t} = \frac{1}{r} \frac{\partial}{\partial r} \left[r D(C) \frac{\partial C}{\partial r} \right] \quad 0 < r < R, \quad 0 < t \leq 1800 \text{ s}, \quad (5)$$

其中水分扩散系数采用题目给定的经验公式:

$$D(C) = 7 \times 10^{-9} \exp \left(-\frac{0.89}{C} \right) \text{ m}^2/\text{s}. \quad (6)$$

初始时刻, 药材含水率均匀分布, 即 $C(r, 0) = 2.55$ 。令 $C_s = C(R, t)$ 。药材表面向外的扩散通量 J_{out} 为 $J_{\text{out}} = -D(C_s) \frac{\partial C}{\partial r} \Big|_{r=R}$ 。根据线性对流传质定律, 表面向环境传出的水分通量为 $h_m(C_s - C_\infty)$, 故表面传质边界为

$$-D(C_s) \frac{\partial C}{\partial r} \Big|_{r=R} = h_m [C_s - C_\infty(t)]. \quad (7)$$

5.1.4 数值离散与求解

令径向步长为 $\Delta r = 0.1 \text{ cm}$, 时间步长为 $\Delta t = 1 \text{ s}$, 径向计算节点定义为:

$$\Delta r = 0.1 \text{ cm} = 1.0 \times 10^{-3} \text{ m}, \quad r_i = i\Delta r, \quad i = 0, 1, \dots, 20 \quad (8)$$

时间层定义为:

$$\Delta t = 1 \text{ s}, \quad t_n = n\Delta t \quad (9)$$

其中, n 为时间层编号。

温度场和水分场均采用径向有限体积法离散, 控制体取同心圆柱薄壳。为统一表示两个扩散方程, 将其写为:

$$s \frac{\partial \phi}{\partial t} = \frac{1}{r} \frac{\partial}{\partial r} \left(r \Gamma \frac{\partial \phi}{\partial r} \right) \quad (10)$$

其中, 变量和参数对应关系为:

$$(\phi, s, \Gamma) = \begin{cases} (T, \rho c_p, k), & \text{温度场,} \\ (C, 1, D(C)), & \text{水分场} \end{cases} \quad (11)$$

在每个时间层 $t_n \rightarrow t_{n+1}$ 内, 时间导数采用后向 Euler 全隐式离散, 即 $\partial \phi / \partial t \approx (\phi^{n+1} - \phi^n) / \Delta t$, 离散方程中的未知量及传递系数均取 $n+1$ 时刻。由式(10)和式(11)在

各径向控制体上的守恒积分，分别得到如下节点离散格式。

内部节点离散 对内部节点 $i = 1, 2, \dots, N-1$ ，控制体的径向权重记为 $M_i = r_i \Delta r$ ，并令 $r_{i+\frac{1}{2}} = r_i + \Delta r/2$ 。对统一控制方程进行积分并采用后向 Euler 格式，得到

$$s_i M_i \frac{\phi_i^{n+1} - \phi_i^n}{\Delta t} = r_{i+\frac{1}{2}} \Gamma_{i+\frac{1}{2}}^{n+1} \frac{\phi_{i+1}^{n+1} - \phi_i^{n+1}}{\Delta r} - r_{i-\frac{1}{2}} \Gamma_{i-\frac{1}{2}}^{n+1} \frac{\phi_i^{n+1} - \phi_{i-1}^{n+1}}{\Delta r} \quad (12)$$

其中 $N = 20$ ， s_i 为节点 r_i 处的储存系数， $\Gamma_{i+\frac{1}{2}}^{n+1}$ 为界面 $r_{i+\frac{1}{2}}$ 处的传递系数。该式分别取 $(\phi, s, \Gamma) = (T, \rho c_p, k)$ 和 $(C, 1, D(C))$ 时，对应温度场和水分场的内部节点离散方程。

中心节点离散 中心控制体为 $0 \leq r \leq \Delta r/2$ 。利用圆柱中心的轴对称无通量条件，中心界面通量为零，从而避免直接计算统一控制方程中的 $1/r$ 项，得到中心节点格式

$$s_0 \frac{\phi_0^{n+1} - \phi_0^n}{\Delta t} = \frac{4\Gamma_{\frac{1}{2}}^{n+1}}{\Delta r^2} (\phi_1^{n+1} - \phi_0^{n+1}) \quad (13)$$

表面节点离散 记表面节点为 $N = 20$ ，表面半控制体的径向权重为

$$M_N = \frac{R^2 - (R - \Delta r/2)^2}{2} \quad (14)$$

将前文的表面 Robin 边界通量式(4)和式(7)代入表面控制体守恒式，得到统一的表面节点离散方程

$$s M_N \frac{\phi_N^{n+1} - \phi_N^n}{\Delta t} = Rb (\phi_\infty^{n+1} - \phi_N^{n+1}) - r_{N-\frac{1}{2}} \Gamma_{N-\frac{1}{2}}^{n+1} \frac{\phi_N^{n+1} - \phi_{N-1}^{n+1}}{\Delta r} \quad (15)$$

其中 ϕ_∞^{n+1} 表示相应的环境温度或环境含水率，比例系数 b 为

$$b = \begin{cases} h, & \phi = T, \\ h_m, & \phi = C \end{cases} \quad (16)$$

非线性扩散系数处理 由于水分扩散系数 $D(C)$ 随含水率 C 非线性变化，水分场控制方程为非线性偏微分方程，故采用 Picard 迭代求解。温度方程中的 ρ 、 c_p 和 k 为常数，每个时间步可直接组装并求解一次线性三对角方程组。取上一时间层解 C^n 作为初始迭代值，在第 m 次迭代中根据 $C_i^{(m)}$ 计算节点扩散系数

$$D_i^{(m)} = 7 \times 10^{-9} \exp \left(-\frac{0.89}{C_i^{(m)}} \right) \quad (17)$$

界面扩散系数采用相邻节点扩散系数的调和平均

$$D_{i+\frac{1}{2}}^{(m)} = \frac{2D_i^{(m)}D_{i+1}^{(m)}}{D_i^{(m)} + D_{i+1}^{(m)}} \quad (18)$$

在固定 $D^{(m)}$ 的条件下求解线性离散方程，得到新的含水率场 $C^{(m+1)}$ 。当相邻两次迭代的最大绝对增量满足

$$\max_i \left| C_i^{(m+1)} - C_i^{(m)} \right| < 10^{-10} \quad (19)$$

时，认为当前时间层收敛，并将收敛解作为下一时间层的初值。

5.1.5 模型计算结果

数值计算覆盖 $t = 0, 1, \dots, 1800 \text{ s}$ 的全部时间层和 $r = 0, 0.1, \dots, 2.0 \text{ cm}$ 的全部径向节点，完整计算结果保存于文件 `result1.xlsx`。从数值解中提取 $t = 100, 300, 600, 900, 1200, 1500, 1800 \text{ s}$ 时， $r = 0, 0.5, 1.0, 1.5, 2.0 \text{ cm}$ 处的温度和水分浓度，结果分别列于表 2 和表 3。

表 2 30 分钟内药材的温度

时间/s	到药材中心的距离/cm				
	0	0.5	1	1.5	2
100	28.0001	28.0003	28.0040	28.0327	28.1801
300	28.0408	28.0635	28.1514	28.3681	28.8489
600	28.4533	28.5360	28.8039	29.3159	30.1652
900	29.3243	29.4583	29.8755	30.6161	31.7304
1200	30.5427	30.7098	31.2223	32.1126	33.4276
1500	31.9957	32.1867	32.7660	33.7463	35.1203
1800	33.5753	33.7720	34.3642	35.3621	36.7856

表 3 30 分钟内药材的水分浓度

时间/s	到药材中心的距离/cm				
	0	0.5	1	1.5	2
100	2.5500	2.5500	2.5500	2.5500	2.2470
300	2.5500	2.5500	2.5500	2.5492	2.0508
600	2.5500	2.5500	2.5500	2.5353	1.8770
900	2.5500	2.5500	2.5497	2.5045	1.7547
1200	2.5500	2.5500	2.5482	2.4646	1.6586
1500	2.5500	2.5499	2.5445	2.4206	1.5787
1800	2.5500	2.5497	2.5383	2.3755	1.5102

图7和图8分别以三维时空曲面和分时刻云图的形式展示了温度场与水分浓度场的分布特征。其中,图 7(a) 和 (b) 分别为温度场和水分浓度场的三维时空分布曲面图;图8(a) 和 (b) 分别为 $t = 0\text{ s}$ 、 900 s 和 1800 s 时刻的温度场和水分浓度场分布云图。

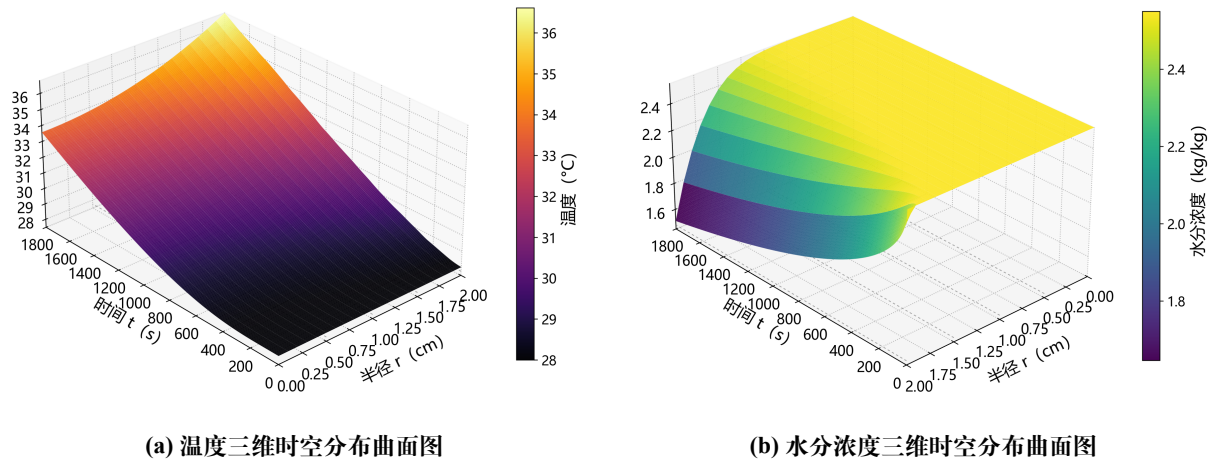


图 7 温度和水分浓度三维时空分布曲面图

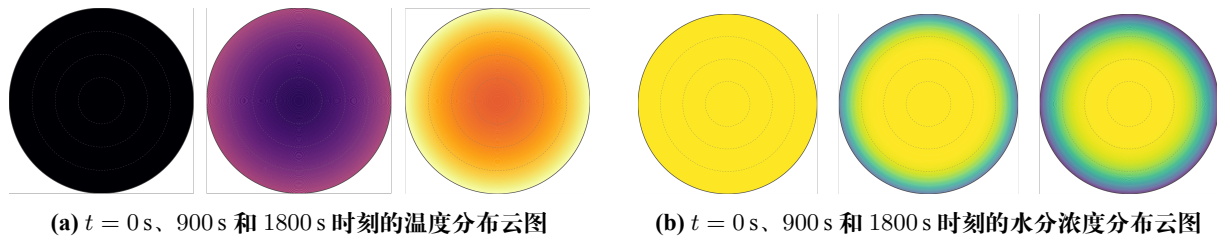


图 8 不同时间的温度和水分浓度分布云图

5.2 问题二模型的建立与求解

5.2.1 问题二模型的更新

问题二沿用问题一建立的一维径向非稳态“传热-传质”模型,进一步考虑药材物性参数随温度和含水率变化的影响。并且,如图9所示,问题二将研究过程拓展到了整个烘干过程,分为预热平衡与恒温干燥阶段。其中,密度、比热容和导热系数随含水率变化,水分扩散系数同时受含水率和温度影响。因此,含水率场通过改变密度、比热容和导热系数影响温度场,温度场又通过扩散系数影响水分场,二者构成双向耦合的关系。

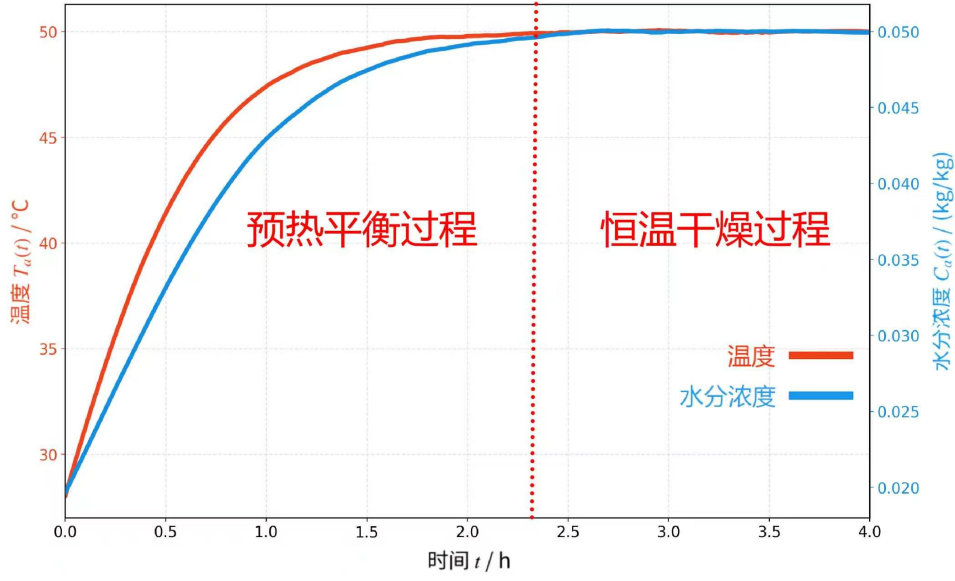


图9 药材预热烘干过程示意图

5.2.2 变物性热湿耦合方程

根据附录3，药材的变物性参数统一表示为

$$\left\{ \begin{array}{l} \rho(C) = 650 + 128C, \\ c_p(C) = 1450 + 2736 \frac{C}{C+1}, \\ k(C) = 0.21 + 0.38 \frac{C}{C+1}, \\ D(C, T_K) = 2.4 \times 10^{-3} \exp\left(-\frac{0.45}{C}\right) \exp\left(-\frac{3850}{T_K}\right) \end{array} \right. \quad (20)$$

其中， ρ 、 c_p 和 k 分别表示密度、比热容和导热系数， D 表示水分扩散系数， T_K 为绝对温度。扩散系数公式中的温度采用绝对温度，因此计算时使用：

$$T_K = T + 273.15 \quad (21)$$

将上述变物性关系代入能量守恒方程和质量守恒方程，得到问题二的温度场控制方程：

$$\boxed{\rho(C)c_p(C)\frac{\partial T}{\partial t} = \frac{1}{r}\frac{\partial}{\partial r}\left[rk(C)\frac{\partial T}{\partial r}\right]} \quad (22)$$

以及水分场控制方程：

$$\boxed{\frac{\partial C}{\partial t} = \frac{1}{r}\frac{\partial}{\partial r}\left[rD(C, T_K)\frac{\partial C}{\partial r}\right]} \quad (23)$$

含水率 C 决定 $\rho(C)$ 、 $c_p(C)$ 和 $k(C)$ ，而温度 T 与含水率 C 共同决定扩散系数

$D(C, T_K)$ 。因此，温度场和水分场需要在同一时间层内迭代求解。

5.2.3 数值离散与耦合求解

非均匀网格的构造思想 药材内部的温度和含水率主要通过表面与烘房交换。表面 Robin 边界直接施加热量和水分通量，烘干初期在表面附近形成比内部更陡的径向梯度。若全域采用能够分辨该边界层的均匀细网格，会在梯度较缓的内部区域产生大量冗余节点；若仅按 0.1 cm 输出间距离散，又不足以准确计算表面梯度和边界通量。为兼顾表面分辨率与计算效率，本文采用“内部较疏、表面加密”的两区节点型有限体积网格。

设药材半径为 $R = 0.02\text{ m}$ ，表面加密层厚度为 $\delta_s = 0.002\text{ m}$ ，内部网格步长为 Δr_b ，表面网格步长为 Δr_s ，加密倍率取 $q = 10$ ，即

$$\Delta r_s = \frac{\Delta r_b}{q}. \quad (24)$$

径向区间总数取 $N = 760$ 。由两区的区间数之和满足

$$\frac{R - \delta_s}{\Delta r_b} + \frac{\delta_s}{\Delta r_s} = N, \quad (25)$$

得到

$$\Delta r_b = \frac{(R - \delta_s) + q\delta_s}{N} = 5.0 \times 10^{-5}\text{ m} = 50\text{ }\mu\text{m}, \quad \Delta r_s = 5.0 \times 10^{-6}\text{ m} = 5\text{ }\mu\text{m}. \quad (26)$$

因此， $0 \leq r \leq 1.8\text{ cm}$ 的内部区域含 360 个区间， $1.8\text{ cm} < r \leq 2.0\text{ cm}$ 的表面层含 400 个区间，共有 $N + 1 = 761$ 个计算节点。节点坐标写为

$$r_i = \begin{cases} i\Delta r_b, & 0 \leq i \leq 360, \\ R - \delta_s + (i - 360)\Delta r_s, & 360 \leq i \leq 760. \end{cases} \quad (27)$$

该网格在仅使用 760 个区间的情况下获得 $5\text{ }\mu\text{m}$ 的表面分辨率；若全域均采用同样的 $5\text{ }\mu\text{m}$ 步长，则需要 4000 个区间。两区网格由此减少约 81% 的空间未知量，同时保留最需要加密的表面区域。题目要求的 $r = 0, 0.1, \dots, 2.0\text{ cm}$ 均为上述网格的实际节点，故结果输出不需要额外空间插值。

非均匀网格上的有限体积离散 记相邻节点中点为控制面

$$r_{i+\frac{1}{2}} = \frac{r_i + r_{i+1}}{2}. \quad (28)$$

约去所有控制体共有的 $2\pi L$ 后，中心、内部和表面节点的径向体积权重分别为

$$M_0 = \frac{r_{\frac{1}{2}}^2}{2}, \quad M_i = \frac{r_{i+\frac{1}{2}}^2 - r_{i-\frac{1}{2}}^2}{2}, \quad M_N = \frac{R^2 - r_{N-\frac{1}{2}}^2}{2}. \quad (29)$$

为统一表示温度场和水分场，定义

$$(\phi, S_i, \Gamma_i, b) = \begin{cases} (T, \rho(C_i)c_p(C_i), k(C_i), h), & \text{温度场,} \\ (C, 1, D(C_i, T_i + 273.15), h_m), & \text{水分场.} \end{cases} \quad (30)$$

界面传递系数采用相邻节点值的调和平均

$$\Gamma_{i+\frac{1}{2}} = \frac{2\Gamma_i\Gamma_{i+1}}{\Gamma_i + \Gamma_{i+1}}, \quad (31)$$

并定义界面导通系数

$$G_{i+\frac{1}{2}} = \frac{r_{i+\frac{1}{2}}\Gamma_{i+\frac{1}{2}}}{r_{i+1} - r_i}. \quad (32)$$

由控制体守恒积分，内部节点 $i = 1, \dots, N-1$ 满足

$$S_i M_i \frac{d\phi_i}{dt} = G_{i+\frac{1}{2}}(\phi_{i+1} - \phi_i) - G_{i-\frac{1}{2}}(\phi_i - \phi_{i-1}). \quad (33)$$

圆心处利用轴对称无通量条件，得到

$$S_0 M_0 \frac{d\phi_0}{dt} = G_{\frac{1}{2}}(\phi_1 - \phi_0), \quad (34)$$

从而不需要在 $r = 0$ 处直接计算 $1/r$ 。表面控制体计入 Robin 边界通量：

$$S_N M_N \frac{d\phi_N}{dt} = Rb(\phi_\infty(t) - \phi_N) - G_{N-\frac{1}{2}}(\phi_N - \phi_{N-1}). \quad (35)$$

上述形式在网格疏密交界面仍保持逐控制体守恒，非均匀间距通过 M_i 和 $G_{i+\frac{1}{2}}$ 直接进入离散方程。

半离散耦合系统与自适应 BDF 将全部温度和含水率节点排列为

$$\mathbf{y} = (T_0, \dots, T_N, C_0, \dots, C_N)^\top \in \mathbb{R}^{1522}, \quad (36)$$

由式(33)–(35)得到刚性非线性常微分方程组

$$\frac{d\mathbf{y}}{dt} = \mathbf{F}(t, \mathbf{y}). \quad (37)$$

每次计算 $\mathbf{F}$ 时, 由当前 $\mathbf{y}$ 同步更新 $\rho(C)$ 、 $c_p(C)$ 、 $k(C)$ 和 $D(C, T_K)$, 再计算热量与水分界面净通量。因此, 第二问是由隐式求解器对 1522 个未知量整体处理其非线性耦合。

时间积分采用自适应变阶 BDF。对第 $n+1$ 个内部时间步, 其一般形式可表示为

$$\sum_{j=0}^{q_n} \alpha_{j,n} \mathbf{y}_{n+1-j} = \Delta t_n \beta_n \mathbf{F}(t_{n+1}, \mathbf{y}_{n+1}), \quad 1 \leq q_n \leq 5, \quad (38)$$

其中阶数 q_n 和内部步长 Δt_n 由局部误差估计自动调整。温度和含水率量纲不同, 故分别设置绝对容差; 主计算参数为

$$\text{rtol} = 2 \times 10^{-9}, \quad \text{atol}_T = 2 \times 10^{-9}, \quad \text{atol}_C = 2 \times 10^{-11}. \quad (39)$$

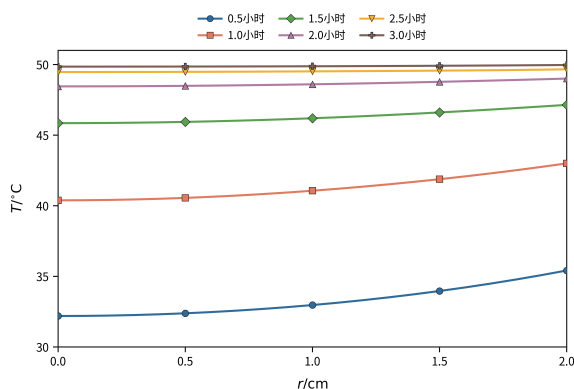
初始内部时间步取 10^{-4} s, 最大内部时间步限制为 5 s。求解器在相邻输出时刻之间可根据误差自动细分时间步。

由于每个节点只与自身及相邻节点的温度、含水率有关, 系统 Jacobian 呈块三对角稀疏结构。计算中预先提供这一稀疏结构, 以降低数值 Jacobian 计算和隐式线性方程组分解的开销, 但不改变控制方程和误差判据。

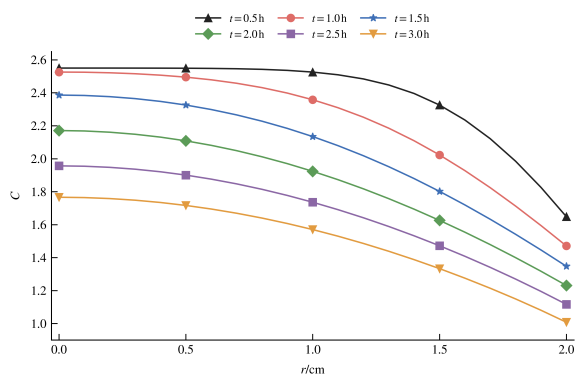
网格与时间误差验证 为检验非均匀网格的充分性, 保持表面加密层厚度和 10 倍加密比例不变, 依次采用 $N = 190, 380, 760$ 进行空间加密。由三层结果得到的温度、含水率最大范数观测阶分别为 1.8790 和 1.9999, 细网格的 1.25 倍 Richardson 安全误差上界分别为 $4.665153 \times 10^{-6} \text{ }^\circ\text{C}$ 和 $1.455194 \times 10^{-5} \text{ kg/kg}$ 。在 $N = 760$ 上进一步将积分容差缩小 4 倍并把最大内部步长减至 2.5 s, 温度和含水率最大变化分别为 $5.360042 \times 10^{-6} \text{ }^\circ\text{C}$ 和 $4.365391 \times 10^{-8} \text{ kg/kg}$ 。结果表明, 表面加密网格和自适应时间积分均已达到题目四位小数输出所需的稳定精度。

5.2.4 模型计算结果

问题二的数值计算覆盖全部时间层 $t = 0, 1, \dots, 10800 \text{ s}$ 和全部径向节点 $r = 0, 0.1, \dots, 2.0 \text{ cm}$ 。基于全部时间层和全部径向节点的数值计算结果, 绘制温度场和水分浓度场在各径向位置上的时间变化折线图, 如图10所示, 其中子图 (a) 为温度场, 子图 (b) 为水分浓度场; 同时提取 0.5, 1.0, 1.5, 2.0, 2.5, 3.0 h 时, $r = 0, 0.5, 1.0, 1.5, 2.0 \text{ cm}$ 处的温度和水分浓度, 结果分别列于表3和表4。



(a) 温度变化折线图



(b) 水分浓度变化折线图

图 10 问题二模型计算结果

表 3 3 小时内药材的温度

时间/h	到药材中心的距离/cm				
	0	0.5	1	1.5	2
0.5	32.1892	32.3821	32.9659	33.9608	35.4130
1.0	40.3816	40.5536	41.0605	41.8782	42.9976
1.5	45.8468	45.9348	46.1930	46.6051	47.1401
2.0	48.4502	48.4881	48.5984	48.7736	49.0033
2.5	49.4670	49.4792	49.5136	49.5653	49.6609
3.0	49.8495	49.8553	49.8746	49.9101	49.9664

表 4 3 小时内药材的水分浓度

时间/h	到药材中心的距离/cm				
	0	0.5	1	1.5	2
0.5	2.5499	2.5489	2.5256	2.3257	1.6486
1.0	2.5257	2.4948	2.3578	2.0230	1.4711
1.5	2.3861	2.3256	2.1344	1.8020	1.3476
2.0	2.1709	2.1084	1.9236	1.6259	1.2311
2.5	1.9566	1.9006	1.7360	1.4720	1.1166
3.0	1.7662	1.7165	1.5702	1.3333	1.0081

5.3 问题三模型的建立与求解

5.3.1 第二问模型的继承与长期边界闭合

问题三不再改变药材的几何尺寸和物性关系，而是在问题二模型的基础上延长计算时间，直至药材全域含水率满足干燥要求。因此，本问仍采用固定半径 $R = 0.02\text{m}$ 的

一维径向模型，温度场和含水率场分别满足式(22)和式(23)，密度、比热容、导热系数和扩散系数均沿用式(20)。

附件 1 给出的烘房环境数据覆盖 $0 \leq t \leq t_a = 14400 \text{ s}$ 。在该区间内保留全部测量点，并按问题一的方法作分段线性插值。题面没有给出 4 h 后的环境数据，而求得全域达标时刻必须延长边界输入。为避免在 t_a 处人为产生跳变，本文声明如下长期闭合：从附件末值出发，在 $\Delta t_a = 60 \text{ s}$ 内线性过渡到稳定烘干环境 50°C 和 0.05 kg/kg ，随后保持不变。令 $\lambda = (t - t_a)/\Delta t_a$ ，则

$$(T_\infty(t), C_\infty(t)) = \begin{cases} (T_{\infty, \text{att}}(t), C_{\infty, \text{att}}(t)), & 0 \leq t \leq t_a, \\ (1 - \lambda)(T_{\infty, \text{att}}(t_a), C_{\infty, \text{att}}(t_a)) + \lambda(50, 0.05), & t_a < t < t_a + \Delta t_a, \\ (50, 0.05), & t \geq t_a + \Delta t_a. \end{cases} \quad (40)$$

式(40)中的过渡段和稳定值属于模型闭合而非实测数据；本文所有第三问主解及辅助算法均使用这一相同输入，避免将边界延拓差异混入算法比较。

5.3.2 烘干结束判据

题目要求烘干结束条件为药材各处的水分浓度均低于 0.15 kg/kg 。定义全域最大含水率

$$C_{\max}(t) = \max_{0 \leq r \leq R} C(r, t), \quad (41)$$

并构造事件函数

$$g(t) = C_{\max}(t) - 0.15. \quad (42)$$

连续临界时刻定义为

$$t_* = \inf \{t > 0 : C_{\max}(t) < 0.15\}. \quad (43)$$

有限体积解采用分段线性空间重构，其单元内最大值必在端点取得，故计算全部节点的最大值即可得到离散重构的全域最大值。计算结果显示，临界阶段的控制位置为圆柱中心 $r = 0$ 。

临界时刻是由场变量导出的决策量，需与场误差分开检查。若阈值附近控制最大值的分支光滑且 $g'(t_*) \neq 0$ ，对扰动后的零点作一阶展开可得

$$\delta t_* \approx -\frac{\delta g(t_*)}{g'(t_*)}. \quad (44)$$

式(44)说明，末期下降斜率较小时，较小的含水率偏差也可能转化为可见的时刻偏差；该式只用于解释局部敏感性，不作为严格误差界。

5.3.3 数值离散与临界时刻求解

固定半径下取无量纲坐标 $\xi = r/R \in [0, 1]$ 。为同时分辨内部缓变区和表层陡峭梯度，主计算使用 $N = 12160$ 个径向区间，其中 $[0, 0.9]$ 布置 5760 个等距区间， $[0.9, 1]$ 布置 6400 个等距区间，即

$$\xi_j = \begin{cases} 0.9j/5760, & 0 \leq j \leq 5760, \\ 0.9 + 0.1(j - 5760)/6400, & 5760 < j \leq 12160. \end{cases} \quad \begin{aligned} \Delta r_{\text{in}} &= 3.125 \mu\text{m}, \\ \Delta r_{\text{surf}} &= 0.3125 \mu\text{m}. \end{aligned} \quad (45)$$

题目要求的 0.1 cm 输出位置均与网格节点对齐。

空间方向沿用式(33)–(35)的守恒有限体积格式；界面导热系数与扩散系数采用正值调和平均

$$a_{i+\frac{1}{2}} = \frac{2a_i a_{i+1}}{a_i + a_{i+1}}, \quad a \in \{k, D\}, \quad (46)$$

从而以单一导通系数计算同一控制面的守恒通量，并保持系数为正。

时间方向首步采用后向 Euler，随后采用可变步长 BDF2[4]。记当前步长 $h_n = t_{n+1} - t_n$ 、步长比 $q = h_n/h_{n-1}$ ，则

$$\left. \frac{\partial \phi}{\partial t} \right|_{t_{n+1}} \approx \frac{1}{h_n} \left[\frac{1+2q}{1+q} \phi^{n+1} - (1+q) \phi^n + \frac{q^2}{1+q} \phi^{n-1} \right]. \quad (47)$$

主时间计划在首分钟取 $h = 0.0078125 \text{ s}$ ，随后以不超过 2 倍的比例逐级放宽，在附件数据及其 60 s 过渡段内最终取 $h = 0.125 \text{ s}$ ，长期阶段最终取 $h = 1 \text{ s}$ 。全部步长均整除 60 s，因而外界数据折点和结果输出点均为时间节点。

温度和含水率在每个时间层内联合 Picard 迭代。第 m 次迭代由候选含水率更新 ρ, c_p, k ，求得温度后再更新 $D(C, T_K)$ 并求含水率。更新量须满足

$$\left\| \Delta T^{(m+1)} - \Delta T^{(m)} \right\|_{\infty} < 2 \times 10^{-12}, \quad \left\| \Delta C^{(m+1)} - \Delta C^{(m)} \right\|_{\infty} < 2 \times 10^{-13}. \quad (48)$$

达到更新量阈值后，程序用最终候选状态重新计算全部物性和通量，并逐控制体核验离散平衡残差；相对残差阈值取 10^{-10} ，另显式计入双精度舍入容差。只有更新量与残差两项均通过时才接受该时间步。

当相邻时间层满足 $g(t_n) \geq 0$ 且 $g(t_{n+1}) < 0$ 时，对最近三个时间层的每个节点构造二次时间插值多项式 $P_i(t)$ ，并定义离散事件函数

$$g_h(t) = \max_{0 \leq i \leq N} P_i(t) - 0.15. \quad (49)$$

在末个时间步内对 g_h 作二分搜索，直至严格下穿区间 $[t_{h,-}, t_{h,+}]$ 满足

$$g_h(t_{h,-}) \geq 0, \quad g_h(t_{h,+}) < 0, \quad t_{h,+} - t_{h,-} < 10^{-4} \text{ s}. \quad (50)$$

该宽度只是所选离散解的求根分辨率，不等于偏微分方程离散误差。每个 Picard 子步产生温度、含水率各一个三对角方程组，均用 Thomas 算法求解；网格几何量和工作数组的复用只影响效率，不改变数值格式。

5.3.4 模型计算结果

主计算共接受 320842 个隐式时间步。严格下穿区间为

$$t_{h,-} = 206900.236572266 \text{ s}, \quad t_{h,+} = 206900.236633301 \text{ s}. \quad (51)$$

在 $t_{h,+}$ 处，全网格未舍入最大含水率为 $0.149999999991 \text{ kg/kg}$ ，控制节点为圆心，圆心下降斜率为 $-2.912878 \times 10^{-7} \text{ kg}/(\text{kg s})$ 。下文空间、时间和事件定位给出的临界时刻综合经验误差估计为 $\hat{\varepsilon}_t = 0.035503 \text{ s}$ ，故向上保留四位小数的工艺报告值取

$$t_{\text{rep}} = 10^{-4} \left\lceil 10^4 \frac{t_{h,+} + \hat{\varepsilon}_t}{3600} \right\rceil = 57.4723 \text{ h}. \quad (52)$$

该取值在当前模型和经验误差估计下留有约 $7.86 \times 10^{-3} \text{ s}$ 的舍入余量。典型时刻的径向含水率见表5。

表 5 药材烘干过程的水分浓度

时间/h	到药材中心的距离/cm				
	0	0.5	1.0	1.5	2.0
6	1.0170	0.9881	0.9015	0.7550	0.5328
12	0.4561	0.4432	0.4031	0.3297	0.1642
18	0.2988	0.2915	0.2687	0.2253	0.0845
24	0.2378	0.2327	0.2167	0.1855	0.0665
30	0.2058	0.2018	0.1892	0.1641	0.0598
36	0.1859	0.1825	0.1718	0.1504	0.0566
42	0.1721	0.1691	0.1597	0.1407	0.0549
48	0.1618	0.1592	0.1507	0.1334	0.0537
54	0.1539	0.1514	0.1436	0.1277	0.0530
57.4723	0.1500	0.1477	0.1402	0.1249	0.0527

由表5可见，任一时刻的含水率均由中心向表面递减，表明水分从内部持续向表面迁移。表面含水率较早接近环境水平，而中心含水率下降最慢，故最终烘干时长由中心

位置控制。末行中心值显示为 0.1500 是四位小数舍入结果，事件判断使用的是未舍入双精度数值。

5.4 问题四模型的建立与求解

5.4.1 均匀仿射收缩与固定域变换

本问先建立一个直接利用附件 2 半径数据的主模型。设药材在时刻 t 的半径为 $R(t)$ ，长度保持初值 $L_0 = 0.25 \text{ m}$ 。附件 2 共有 145 个测点，覆盖 $0 \leq t \leq 72 \text{ h}$ ；相邻测点间只作局部分段线性插值

$$R(t) = R_j + \frac{t - t_j}{t_{j+1} - t_j} (R_{j+1} - R_j), \quad t_j \leq t \leq t_{j+1}. \quad (53)$$

式(53)仅用于恢复离散测量之间的连续边界，并不假定 72 h 内的整体收缩规律为线性。本文所得临界时刻位于附件 2 观测区间内，因而不需要外推半径。

均匀仿射收缩是指每个材料层保持相同的径向缩放比。若材料层的初始半径为 r_0 ，则

$$r(t) = \lambda(t)r_0, \quad \lambda(t) = \frac{R(t)}{R_0}. \quad (54)$$

因此不同材料层不会相互穿越，初始同心圆在收缩后仍为同心圆。取无量纲材料坐标

$$\xi = \frac{r}{R(t)} = \frac{r_0}{R_0}, \quad 0 \leq \xi \leq 1, \quad (55)$$

并记 $\Theta(\xi, t) = T(\xi R(t), t)$ 、 $W(\xi, t) = C(\xi R(t), t)$ 。由式(54)，材料骨架速度为 $u_r = (\dot{R}/R)r$ ；对任意随骨架运动的场 f ，链式法则给出

$$\frac{Df}{Dt} = \left. \frac{\partial f}{\partial t} \right|_r + u_r \frac{\partial f}{\partial r} = \left. \frac{\partial \hat{f}}{\partial t} \right|_\xi. \quad (56)$$

这表明采用材料坐标后，网格运动项已经包含在式(56)中，不应重复添加收缩对流项。

5.4.2 收缩坐标下的热湿耦合模型

主模型沿用前两问的一维径向传热传质结构，并从 $t = 0$ 起采用附录 4 给出的物性关系。圆柱径向扩散算子在 $r = \xi R(t)$ 下产生尺度因子 $R(t)^{-2}$ ，故固定区间 $\xi \in [0, 1]$ 上的控制方程为

$$\boxed{\rho_w(W)c_p(W)\frac{\partial \Theta}{\partial t} = \frac{1}{R(t)^2\xi}\frac{\partial}{\partial \xi}\left[\xi k(W)\frac{\partial \Theta}{\partial \xi}\right]}, \quad (57)$$

$$\frac{\partial W}{\partial t} = \frac{1}{R(t)^2 \xi} \frac{\partial}{\partial \xi} \left[\xi D(W, \Theta_K) \frac{\partial W}{\partial \xi} \right], \quad (58)$$

其中 $\Theta_K = \Theta + 273.15$, 且

$$\begin{cases} \rho_w(W) = 760 + 90W, \\ c_p(W) = 1850 + 2150 \frac{W}{1+W}, \\ k(W) = 0.12 + 0.20 \frac{W}{1+W}, \\ D(W, \Theta_K) = 4.2 \times 10^{-4} \exp\left(-\frac{0.30}{W} - \frac{3850}{\Theta_K}\right). \end{cases} \quad (59)$$

式(59)依次采用 kg/m^3 、 $\text{J}/(\text{kg} \cdot \text{K})$ 、 $\text{W}/(\text{m} \cdot \text{K})$ 和 m^2/s 为单位。

初始条件为 $\Theta(\xi, 0) = 28^\circ\text{C}$ 、 $W(\xi, 0) = 2.55$ 。圆心满足对称条件 $\Theta_\xi(0, t) = W_\xi(0, t) = 0$ ；移动表面的第三类边界条件为

$$\frac{k(W_s)}{R(t)} \Theta_\xi(1, t) = h(T_\infty - \Theta_s), \quad \frac{D(W_s, \Theta_{K,s})}{R(t)} W_\xi(1, t) = h_m(C_\infty - W_s). \quad (60)$$

其中 $h = 25 \text{ W}/(\text{m}^2 \cdot \text{K})$ 、 $h_m = 8 \times 10^{-7} \text{ m/s}$ 。环境过程沿用式(40)，从而保证第三、四问长期边界的比较口径一致。

空间离散沿用前文的守恒有限体积思想，在 $[0, 0.9]$ 与 $[0.9, 1]$ 采用步长比为 10 : 1 的表层加密网格。正式计算取 $N = 6080$ 个径向区间，使用隐式 BDF 方法 [4] 推进温湿耦合状态；相对容差为 5×10^{-12} ，温度和含水率绝对容差分别为 5×10^{-12} 和 5×10^{-14} 。定义

$$C_{\max}^{(4)}(t) = \max_{0 \leq \xi \leq 1} W(\xi, t), \quad t_*^{(4)} = \inf\{t > 0 : C_{\max}^{(4)}(t) < 0.15\}, \quad (61)$$

并利用稠密输出在相邻保存时刻之间定位 $C_{\max}^{(4)} = 0.15$ 的零点。全部输出时刻的最大含水率均位于圆柱中心。

5.4.3 主模型计算结果

数值计算得到连续临界时刻

$$t_*^{(4)} = 183926.094367684 \text{ s} = 51.0905817688 \text{ h} \approx 51.09 \text{ h}. \quad (62)$$

由于临界时刻处 $C_{\max}^{(4)} = 0.15$ ，严格不等式在任意略晚于 $t_*^{(4)}$ 的时刻成立。典型时刻的含水率和实测半径见表6。

表 6 均匀仿射收缩主模型的含水率计算结果

时间/h	到药材中心的距离/cm				当前半径/cm
	0	0.5	1.0	药材表面	
6	1.7196	1.5377	1.0226	0.4207	1.3740
12	0.7377	0.6546	0.4083	0.1670	1.2480
18	0.4087	0.3686	0.2397	0.0892	1.2140
24	0.2851	0.2611	0.1790	0.0673	1.2040
30	0.2263	0.2094	0.1493	0.0595	1.2010
36	0.1928	0.1797	0.1317	0.0560	1.2000
42	0.1712	0.1604	0.1200	0.0541	1.2000
48	0.1561	0.1468	0.1116	0.0530	1.2000
51.0906	0.1500	0.1413	0.1081	0.0526	1.2000

与问题三固定半径模型的 57.4723 h 相比，主模型的临界时刻缩短约 6.3817 h。该差异同时包含半径收缩和附录 4 物性变化，不能全部解释为单一的几何收缩效应；就方程结构而言，半径减小通过式(57)和式(58)中的 $R(t)^{-2}$ 加快了内部传递。

六、模型检验及合理性分析

6.1 第三问模型的数值检验

6.1.1 空间-时间收敛性

主方法的空间检验固定时间计划，采用 $N = 3040, 6080, 12160$ 三层相似加密网格；时间检验固定 $N = 12160$ ，将“初始/附件期/长期”三段最大步长分别取为 $(0.03125, 0.5, 4)$ 、 $(0.015625, 0.25, 2)$ 和 $(0.0078125, 0.125, 1)$ s。全部场差异在 60 s 至 206880 s 的共同输出时段、281 个共同随体位置上计算，临界时刻则单独比较。

对三级结果 $Q_h, Q_{h/2}, Q_{h/4}$ ，观测阶与带 1.5 安全系数的细解余量估计定义为 [6]

$$p_{\text{obs}} = \log_2 \frac{|Q_h - Q_{h/2}|}{|Q_{h/2} - Q_{h/4}|}, \quad \hat{\varepsilon}_Q = 1.5 \frac{|Q_{h/2} - Q_{h/4}|}{2^{p_{\text{obs}}} - 1}. \quad (63)$$

实际三级差值见表7。

表 7 第三问主方法的独立空间、时间加密结果

指标与加密方向	粗-中差异	中-细差异	p_{obs}	细解余量估计
T , 空间	$1.42505 \times 10^{-7} \text{ } ^\circ\text{C}$	$3.56262 \times 10^{-8} \text{ } ^\circ\text{C}$	2.000001	$1.78131 \times 10^{-8} \text{ } ^\circ\text{C}$
T , 时间	$5.19787 \times 10^{-7} \text{ } ^\circ\text{C}$	$1.29712 \times 10^{-7} \text{ } ^\circ\text{C}$	2.002610	$6.48559 \times 10^{-8} \text{ } ^\circ\text{C}$
C , 空间	5.17710×10^{-6}	1.29271×10^{-6}	2.001742	6.46356×10^{-7}
C , 时间	4.12778×10^{-8}	1.03212×10^{-8}	1.999759	5.16174×10^{-9}
t_* , 空间	0.282288 s	0.0705566 s	2.000312	0.0352783 s
t_* , 时间	$1.15967 \times 10^{-3} \text{ s}$	$3.05176 \times 10^{-4} \text{ s}$	1.925999	$1.63487 \times 10^{-4} \text{ s}$

空间和时间加密均呈接近二阶的响应。将空间、时间估计相加，得到温度和含水率在上述共同采样集上的综合经验误差估计

$$\hat{\varepsilon}_T = 8.26690 \times 10^{-8} \text{ }^\circ\text{C}, \quad \hat{\varepsilon}_C = 6.51518 \times 10^{-7} \text{ kg/kg}. \quad (64)$$

临界时刻再计入 $6.10352 \times 10^{-5} \text{ s}$ 的事件区间宽度，得到

$$\hat{\varepsilon}_t = 0.0352783 + 0.000163487 + 0.0000610352 = 0.035503 \text{ s}. \quad (65)$$

这些量依赖式(63)的渐近展开假设，是保守的工程误差估计而非严格数学上界。主解全部时间步均通过最终状态残差检查，最大热量、水分离散平衡相对残差分别为 1.52×10^{-11} 和 2.03×10^{-14} ；这些残差只验证所选离散方程的收支，不代表完整热力学模型已经得到实验验证。

6.1.2 跨算法一致性

三级自收敛回答“同一有限体积离散继续加密时结果如何变化”，但不能排除各级网格共同继承的实现偏差。为此，本文采用分区 Chebyshev-Lobatto 弱形式谱元 [5] 和五阶 Radau IIA 方法 [4] 重新求解相同的径向初边值问题。谱元使用 $s = (r/R)^2$ 坐标、9 个径向子区间；最高阶算例每区采用 64 阶多项式，共 577 个场自由度。它在空间近似、边界施加、时间推进以及空间最大值搜索方面均不同于有限体积主方法。

谱元的 24、40、64 阶加密中，含水率最大采样差由 1.93020×10^{-7} 降至 $2.67740 \times 10^{-8} \text{ kg/kg}$ ，相邻临界时刻差分别为 $1.70841 \times 10^{-5} \text{ s}$ 和 $1.01806 \times 10^{-5} \text{ s}$ ；在 64 阶空间离散上收紧 Radau 容差后，含水率最大采样差为 $1.51519 \times 10^{-11} \text{ kg/kg}$ ，临界时刻差为 $3.10441 \times 10^{-5} \text{ s}$ 。这些是共同采样集上的观测差异，不是严格误差界。

本文进一步从保存的全精度结果中，对有限体积与 64 阶谱元在 3448 个公共分钟、题目要求的 21 个实际径向位置上重新计算差异，得到

$$\begin{aligned} E_T^{\text{FVM,SEM}} &= 4.57080 \times 10^{-8} \text{ }^\circ\text{C}, \\ E_C^{\text{FVM,SEM}} &= 4.67183 \times 10^{-8} \text{ kg/kg}, \\ E_t^{\text{FVM,SEM}} &= 0.0233898 \text{ s}. \end{aligned} \quad (66)$$

谱元临界时刻为 206900.213243 s，与有限体积结果的差异小于式(65)的经验估计。由此可认为，第三问的径向场分布及四位小数时长没有表现出明显的单一空间离散或时间推进依赖；这一结论仍只属于共同模型下的数值交叉证据，不能替代物理实验验证。综上，有限体积自收敛与谱元同模型复算分别提供了数值稳定性和跨离散一致性证据，第三问正式报告 $t_{\text{rep}} = 57.4723 \text{ h}$ 。

6.2 第四问主模型的干质量守恒检验

6.2.1 干质量检验量的构造

主模型已经给出清晰的数值答案，但还需检查其几何收缩与附录 4 真实湿密度能否共同满足物质守恒。干基含水率的定义为 $C = dm_w/dm_d$ ，故 $dm_{wet} = (1 + C)dm_d$ 。若附录 4 的 $\rho_w(C) = 760 + 90C$ 是当前湿体积的真实密度，则单位当前体积中的干固体质量必为

$$\boxed{\rho_d(C) = \frac{\rho_w(C)}{1 + C} = \frac{760 + 90C}{1 + C}.} \quad (67)$$

式(67)只是含水率与体积密度定义的恒等变换，并非新的经验假设。初始状态均匀，因此

$$\rho_{d0} = \frac{760 + 90C_0}{1 + C_0} = 278.7323944 \text{ kg/m}^3, \quad M_{d0} = \pi R_0^2 L_0 \rho_{d0} = 0.08756636 \text{ kg}. \quad (68)$$

把主模型计算出的 $W(\xi, t)$ 、实测半径 $R(t)$ 和固定长度 L_0 代入真实密度定义，可得到后验干质量检验量

$$M_d^*(t) = 2\pi L_0 R(t)^2 \int_0^1 \frac{760 + 90W(\xi, t)}{1 + W(\xi, t)} \xi d\xi. \quad (69)$$

在式(62)的临界时刻，数值积分得到 $M_d^*(t_*^{(4)})/M_{d0} \approx 0.8903$ ，即相对初值减少约 10.97%。这并不表示真实药材必然损失了 10.97% 的干物质，而是说明主模型的“实测半径、固定长度、仿射收缩和真实湿密度”不能共同代表同一个严格守恒的材料运动。

6.2.2 矛盾并非由数值离散造成

上述结论还可不依赖主模型的数值场而直接证明。由式(67)，对 $C \geq 0$ 恒有 $\rho_d(C) \leq 760 \text{ kg/m}^3$ 。若总干质量守恒且长度不超过初始长度，则

$$M_{d0} = \int_{\Omega(t)} \rho_d dV \leq 760\pi R(t)^2 L(t) \leq 760\pi R(t)^2 L_0, \quad (70)$$

从而必须满足

$$\boxed{R(t) \geq R_0 \sqrt{\frac{\rho_{d0}}{760}} = 1.211203 \text{ cm}.} \quad (71)$$

然而附件 2 在 24 h 给出的半径已为 1.204 cm，在 72 h 进一步减至 1.198 cm，均小于式(71)。因此，真正不能完全相容的是

$$\boxed{\text{真实湿密度} + \text{实测外半径} + \text{干质量守恒} + \text{长度不增加} .}$$

该必要条件没有使用仿射映射或扩散方程，所以仅取消均匀仿射收缩、细化网格或改变临界时刻求解器，都不能消除这一结构矛盾。

6.3 真实湿密度下的两个守恒对照模型

为判断应放松哪一项几何约束，本文进一步构造两个非仿射守恒模型。两者均保留附录 4 真实湿密度和严格干质量守恒，但分别放松“长度固定”和“外半径必须作为输入”。

6.3.1 共同的累计干质量坐标

令 $\mu = m_d/M_d \in [0, 1]$ 为归一化累计干质量坐标，则固定的 μ 区间始终代表同一部分干固体。对当前圆柱环层有

$$M_d d\mu = 2\pi L(t)\rho_d(C)r dr, \quad (72)$$

故逐层几何映射和总体积约束分别为

$$\frac{\partial r^2}{\partial \mu} = \frac{M_d}{\pi L(t)\rho_d(C)}, \quad \pi R(t)^2 L(t) = M_d \int_0^1 \frac{1}{\rho_d(C)} d\mu. \quad (73)$$

式(72)把局部干质量守恒直接写入材料坐标。依据 Fick 定律和 Fourier 定律 [1, 2, 3]，两个对照模型共同采用

$$\frac{\partial C}{\partial t} = \frac{\partial}{\partial \mu} \left(K_C \frac{\partial C}{\partial \mu} \right), \quad (1 + C)c_p(C) \frac{\partial T}{\partial t} = \frac{\partial}{\partial \mu} \left(K_T \frac{\partial T}{\partial \mu} \right), \quad (74)$$

其中

$$K_C = \frac{4\pi^2 L(t)^2 r^2 \rho_d(C)^2 D}{M_d^2}, \quad K_T = \frac{4\pi^2 L(t)^2 r^2 \rho_d(C) k}{M_d^2}. \quad (75)$$

初始条件、环境过程及表面对流系数与主模型相同。式(74)采用单位干质量形式，因而无需另设所谓“收缩浓缩项”。

6.3.2 对照模型一：输入实测半径，反演长度

第一个对照模型保留附件 2 的 $R_{\text{obs}}(t)$ ，但不再固定长度。由式(73)得到

$$L(t) = \frac{M_d}{\pi R_{\text{obs}}(t)^2} \int_0^1 \frac{1}{\rho_d(C(\mu, t))} d\mu, \quad (76)$$

以及非仿射径向映射

$$r^2(\mu, t) = R_{\text{obs}}(t)^2 \frac{\int_0^\mu \rho_d(C)^{-1} d\eta}{\int_0^1 \rho_d(C)^{-1} d\eta}. \quad (77)$$

以 $N = 3040$ 、 $\Delta t = 1\text{ s}$ 计算得到临界时刻 47.8764 h；反演长度由 25 cm 最高增至 32.3855 cm，临界时刻仍为 28.0723 cm。该方案在数学上同时满足真实密度、实测半径和逐环干质量守恒，但代价是产生了未经观测支持的显著轴向伸长。由于题目没有给出长度时序，反演长度只能视为“使三项条件相容所需的等效几何量”，不能直接宣称为药材的真实变形。

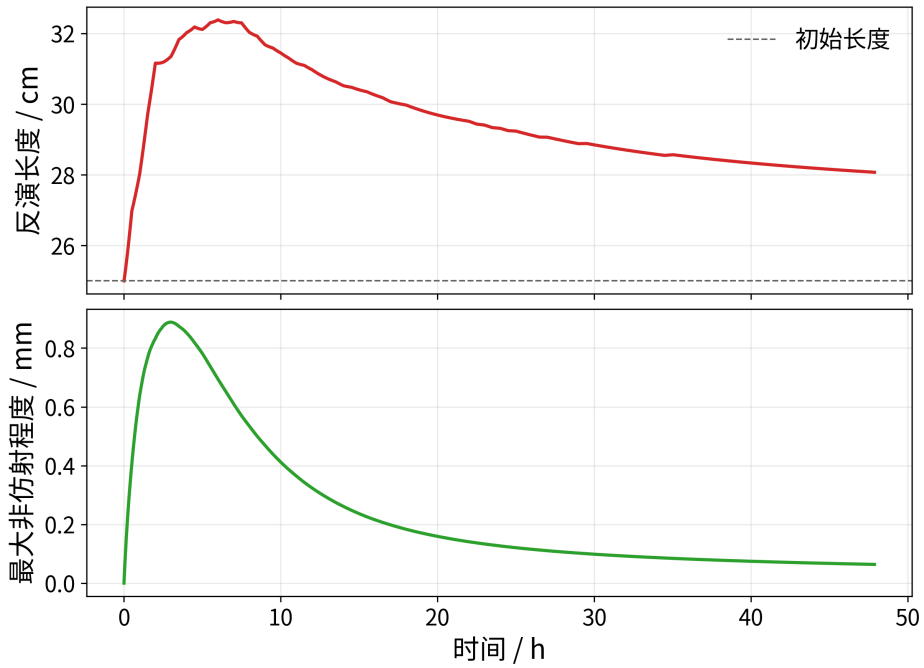


图 11 输入实测半径后由于质量守恒反演得到的长度及非仿射程度

6.3.3 对照模型二：固定长度，独立预测半径

第二个对照模型固定 $L = L_0$ ，不把附件 2 半径输入正向求解。外半径由式(73)唯一预测为

$$R_{\text{pred}}(t) = R_0 \left[\rho_{d0} \int_0^1 \frac{1}{\rho_d(C(\mu, t))} d\mu \right]^{1/2}. \quad (78)$$

附件 2 只在求解完成后用于独立验证。参考计算得到 $R_{\text{pred}}(72\text{ h}) = 1.264861\text{ cm}$ ，而实测值为 1.198000 cm；145 个测量时刻的 $\text{MAE} = 0.109782\text{ cm}$ 、 $\text{RMSE} = 0.120537\text{ cm}$ ，除初始点外预测残差全部为正，最大误差 0.229165 cm 出现在 6 h。该模型的临界时刻为 56.2103 h。

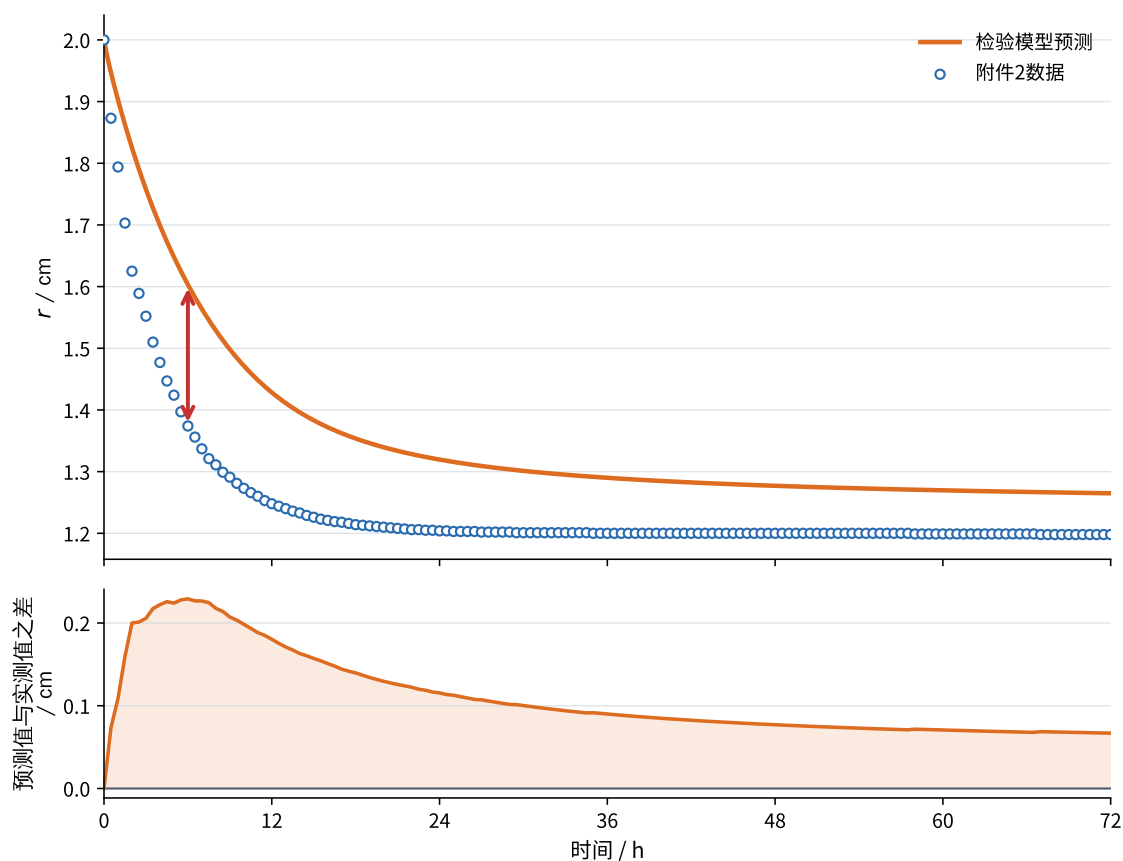


图 12 固定长度守恒模型的半径预测与附件 2 实测值比较

图12显示，模型预测曲线在整个非初始观测区间均高于实测曲线，说明固定长度、真实湿密度和干质量守恒虽然彼此数学相容，但所预测的体积收缩不足。也就是说，该方案的弊端不是无法求解，而是不能充分复现实测收缩过程。

6.4 检验结果与主答案取舍

三种模型的作用和结论汇总于表8。两个守恒对照不是为了从三个时长中挑选一个更“好看”的结果，而是分别检验放松长度约束和放松半径输入后会发生什么。

表 8 第四问三种闭合方式的结果与解释边界

方案	临界时刻	主要证据	主要局限
均匀仿射收缩主模型	51.0906 h	直接采用附件 2 半径，完成题目要求的移动边界计算	若密度为真实湿密度，临界时刻干质量检验量约减少 10.97%
实测半径-反演长度	47.8764 h	真实密度、实测半径和逐环干质量同时守恒	反演长度最高 32.3855 cm，缺少轴向实测支持
固定长度-预测半径	56.2103 h	附件 2 未进入正向求解，可作独立外部检验	半径 RMSE 为 0.120537 cm，系统性高估收缩后半径

为排除“主结果只是离散误差”的可能性，本文对主模型使用 $N = 1520, 3040, 6080$

三层相似网格。相邻两次加密的含水率最大差分别为 6.945758×10^{-6} 和 $1.736444 \times 10^{-6} \text{ kg/kg}$ ，观测收敛阶为 1.999997；带 1.25 安全系数的细网格 Richardson 经验误差估计为 $7.235205 \times 10^{-7} \text{ kg/kg}$ [6]。在 $N = 6080$ 上进一步收紧时间积分后，临界时刻仅变化 $5.47 \times 10^{-6} \text{ s}$ 。因此 51.09 h 在给定主模型内部具有充分的数值稳定性，但这种数值收敛不能消除式(71)揭示的模型结构矛盾。

综合题目作答目标，本文仍将直接使用附件 2 并采用均匀仿射收缩所得的

$$t_*^{(4)} \approx 51.09 \text{ h}$$

作为第四问正式答案；同时明确该结果是均匀仿射主模型下的工程基线，而非已经同时通过真实湿密度、几何观测和干质量守恒验证的唯一物理真值。另两个时长是结构敏感性结果，不能解释为统计置信区间，也不替代主答案。

七、模型评价与展望

7.1 模型优点

- **由简入繁，主次清晰：**先用均匀仿射收缩完成题目要求并得到 51.09 h，再由守恒检验决定是否需要更复杂的几何描述，避免一开始引入无法辨识的自由度。
- **物理检验可追溯：**干质量矛盾由含水率定义、真实湿密度和圆柱体积直接推出；累计干质量坐标又将逐环守恒写成几何恒等式，结论不依赖单一求解器的偶然输出。
- **区分数值误差与结构误差：**主模型完成网格和时间加密；固定长度对照的半径数值差值约为 10^{-6} cm 量级，而模型-数据偏差约为 10^{-1} cm 量级，因此能够判断主要问题来自闭合假设而非网格精度。
- **对照模型具有可解释性：**一个模型把冲突显化为异常的反演长度，另一个模型把冲突显化为可直接与附件 2 比较的半径偏差，使每项额外假设的代价都可以被观察和讨论。

7.2 模型局限

- 主模型直接利用实测半径，但在固定长度与真实湿密度解释下不能严格保持干质量，因此 51.09 h 带有明确的结构条件。
- 反演长度模型缺少长度实测约束，预测出的轴向伸长不能视为已经验证的真实变形；固定长度模型虽严格守恒，却系统性高估附件 2 半径。
- 三种模型均以一维径向场为主，尚未联合描述端面传热传质、轴向应变、孔隙率变化、组织压实与开裂等可能影响宏观体积的机制。

7.3 进一步改进方向

后续实验应同步测量 $R(t)$ 、 $L(t)$ 、样品总质量及代表时刻的真实体积密度或孔隙率。只需增加独立的长度或体积信息，便可判断偏差主要来自轴向变形还是密度-孔隙关系。数据充分后，可在累计干质量坐标基础上引入轴向伸缩比和孔隙率状态变量，并建立二维轴对称 (r, z) 传热传质模型，对侧面与端面分别标定换热、传质系数。这样才能在严格质量守恒的同时利用半径观测，并把第四问的条件性工程估计进一步提升为可由实验辨识和验证的物理模型。

参考文献

- [1] Fick A. On liquid diffusion[J]. Philosophical Magazine, 1855, 10(63): 30–39. DOI: <https://doi.org/10.1080/14786445508641925>.
- [2] Crank J. The Mathematics of Diffusion[M]. 2nd ed. Oxford: Clarendon Press, 1975.
- [3] Bird R B, Stewart W E, Lightfoot E N. Transport Phenomena[M]. 2nd ed. New York: Wiley, 2002.
- [4] Hairer E, Wanner G. Solving Ordinary Differential Equations II: Stiff and Differential-Algebraic Problems[M]. 2nd ed. Berlin: Springer, 1996. DOI: <https://doi.org/10.1007/978-3-642-05221-7>.
- [5] Trefethen L N. Spectral Methods in MATLAB[M]. Philadelphia: SIAM, 2000. DOI: <https://doi.org/10.1137/1.9780898719598>.
- [6] Roache P J. Verification and Validation in Computational Science and Engineering[M]. Albuquerque: Hermosa Publishers, 1998.

A 问题一及共用核心程序

第一至第四问共用径向有限体积空间离散。文件 1.txt 首先给出物性函数、表层加密网格、守恒通量、隐式时间推进、临界时刻定位和 Richardson 误差估计，随后给出问题一的常物性温度场与非线性水分扩散实现。附件读取、结果写入、日志和绘图程序从略，完整可执行程序见支撑材料。

```
1 # Question 1 and shared finite-volume core routines
2 """Core routines for Questions 1--4 (I/O, logging and plotting omitted)."""
3
4 from __future__ import annotations
5
6 from collections.abc import Callable
7 import numpy as np
8 from scipy.integrate import solve_ivp
9 from scipy.optimize import brentq
10
11
12 HEAT_TRANSFER = 25.0      # W/(m^2 K)
13 MASS_TRANSFER = 8.0e-7   # m/s
14 INITIAL_T = 28.0         # deg C
15 INITIAL_C = 2.55         # kg/kg, dry basis
16 DRY_LIMIT = 0.15        # kg/kg, dry basis
17
18
19 def material_properties(
20     moisture: np.ndarray, temperature_c: np.ndarray, model: int
21 ) -> tuple[np.ndarray, np.ndarray, np.ndarray, np.ndarray]:
22     """Return rho, cp, k and D for the selected appendix model."""
23     c = np.maximum(np.asarray(moisture, dtype=float), 1.0e-12)
24     tk = np.asarray(temperature_c, dtype=float) + 273.15
25     ratio = c / (1.0 + c)
26
27     if model == 1: # Question 1: constant thermal properties
28         rho = np.full_like(c, 820.0)
29         cp = np.full_like(c, 2600.0)
30         k = np.full_like(c, 0.36)
31         diffusivity = 7.0e-9 * np.exp(-0.89 / c)
32     elif model == 3: # Attachment 3, Questions 2 and 3
33         rho = 650.0 + 128.0 * c
34         cp = 1450.0 + 2736.0 * ratio
35         k = 0.21 + 0.38 * ratio
36         diffusivity = 2.4e-3 * np.exp(-0.45 / c - 3850.0 / tk)
37     elif model == 4: # Attachment 4, Question 4
38         rho = 760.0 + 90.0 * c
39         cp = 1850.0 + 2150.0 * ratio
40         k = 0.12 + 0.20 * ratio
41         diffusivity = 4.2e-4 * np.exp(-0.30 / c - 3850.0 / tk)
42     else:
43         raise ValueError("model must be 1, 3 or 4")
44     return rho, cp, k, diffusivity
45
46
47 def refined_grid(intervals: int, surface_fraction: float = 0.1) -> np.ndarray:
48     """Grid on x in [0,1], with a 10:1 refinement in the surface layer."""
49     ratio = 10
50     bulk_fraction = 1.0 - surface_fraction
51     bulk_intervals = round(
52         intervals * bulk_fraction / (bulk_fraction + ratio * surface_fraction)
53     )
54     surface_intervals = intervals - bulk_intervals
```

```

55     bulk = np.linspace(0.0, bulk_fraction, bulk_intervals + 1)
56     surface = np.linspace(
57         bulk_fraction, 1.0, surface_intervals + 1
58     )[1:]
59     return np.concatenate((bulk, surface))
60
61
62 def storage_weights(x: np.ndarray) -> np.ndarray:
63     """Integral of x dx over every node-centred cylindrical control volume."""
64     faces = 0.5 * (x[:-1] + x[1:])
65     weights = np.empty_like(x)
66     weights[0] = 0.5 * faces[0] ** 2
67     weights[1:-1] = 0.5 * (faces[1:] ** 2 - faces[:-1] ** 2)
68     weights[-1] = 0.5 * (1.0 - faces[-1] ** 2)
69     return weights
70
71
72 def harmonic_mean(left: np.ndarray, right: np.ndarray) -> np.ndarray:
73     return 2.0 * left * right / (left + right)
74
75
76 def thomas_solve(
77     lower: np.ndarray, diagonal: np.ndarray,
78     upper: np.ndarray, right: np.ndarray,
79 ) -> np.ndarray:
80     """Solve a tridiagonal system (Thomas algorithm)."""
81     b, d = diagonal.copy(), right.copy()
82     for i in range(1, b.size):
83         factor = lower[i - 1] / b[i - 1]
84         b[i] -= factor * upper[i - 1]
85         d[i] -= factor * d[i - 1]
86     answer = np.empty_like(d)
87     answer[-1] = d[-1] / b[-1]
88     for i in range(b.size - 2, -1, -1):
89         answer[i] = (d[i] - upper[i] * answer[i + 1]) / b[i]
90     return answer
91
92
93 def bdf_history(
94     old: np.ndarray,
95     previous: np.ndarray | None,
96     step_s: float,
97     previous_step_s: float | None = None,
98 ) -> tuple[float, np.ndarray]:
99     """Return a0 and the history term for BE/variable-step BDF2."""
100     if previous is None:
101         return 1.0 / step_s, old / step_s
102     previous_step_s = step_s if previous_step_s is None else previous_step_s
103     q = step_s / previous_step_s
104     a0 = (1.0 + 2.0 * q) / ((1.0 + q) * step_s)
105     history = ((1.0 + q) * old - q**2 * previous / (1.0 + q)) / step_s
106     return a0, history
107
108
109 def implicit_fvm_step(
110     old: np.ndarray,
111     previous: np.ndarray | None,
112     step_s: float,
113     previous_step_s: float | None,
114     x: np.ndarray,
115     radius: float,
116     storage: np.ndarray,
117     coefficient: np.ndarray,
118     transfer: float,

```



```

119     ambient: float,
120 ) -> np.ndarray:
121     """One conservative BE/BDF2 step on the fixed reference interval."""
122     a0, history = bdf_history(old, previous, step_s, previous_step_s)
123     faces = 0.5 * (x[:-1] + x[1:])
124     conductance = (
125         faces * harmonic_mean(coefficient[:-1], coefficient[1:]) / np.diff(x)
126     )
127     capacity = radius**2 * storage_weights(x) * storage
128     diagonal = a0 * capacity
129     right = capacity * history
130     diagonal[:-1] += conductance
131     diagonal[1:] += conductance
132     diagonal[-1] += radius * transfer
133     right[-1] += radius * transfer * ambient
134     return thomas_solve(-conductance, diagonal, -conductance, right)
135
136
137 def picard_bdf2_step(
138     old_t: np.ndarray,
139     old_c: np.ndarray,
140     previous_t: np.ndarray | None,
141     previous_c: np.ndarray | None,
142     step_s: float,
143     previous_step_s: float | None,
144     x: np.ndarray,
145     radius: float,
146     model: int,
147     ambient: tuple[float, float],
148     tolerance: float = 1.0e-10,
149 ) -> tuple[np.ndarray, np.ndarray]:
150     """Coupled Picard update used by the fixed-step solvers in Q1 and Q3."""
151     temperature, moisture = old_t.copy(), old_c.copy()
152     for _ in range(50):
153         rho, cp, k, _ = material_properties(moisture, temperature, model)
154         new_t = implicit_fvm_step(
155             old_t, previous_t, step_s, previous_step_s, x, radius,
156             rho * cp, k, HEAT_TRANSFER, ambient[0],
157         )
158         _, _, _, diffusivity = material_properties(moisture, new_t, model)
159         new_c = implicit_fvm_step(
160             old_c, previous_c, step_s, previous_step_s, x, radius,
161             np.ones_like(moisture), diffusivity, MASS_TRANSFER, ambient[1],
162         )
163         change = max(
164             np.max(np.abs(new_t - temperature)),
165             np.max(np.abs(new_c - moisture)),
166         )
167         temperature, moisture = new_t, new_c
168         if change < tolerance:
169             return temperature, moisture
170     raise RuntimeError("Picard iteration did not converge")
171
172
173 def transport_balance(
174     field: np.ndarray,
175     coefficient: np.ndarray,
176     x: np.ndarray,
177     radius: float,
178     transfer: float,
179     ambient: float,
180 ) -> np.ndarray:
181     """Conservative internal-face fluxes and the surface Robin flux."""
182     faces = 0.5 * (x[:-1] + x[1:])

```

```

183     face_coefficient = harmonic_mean(coefficient[:-1], coefficient[1:])
184     flux = faces * face_coefficient * np.diff(field) / np.diff(x)
185     balance = np.empty_like(field)
186     balance[0] = flux[0]
187     balance[1:-1] = flux[1:] - flux[:-1]
188     balance[-1] = radius * transfer * (ambient - field[-1]) - flux[-1]
189     return balance
190
191
192 def build_rhs(
193     x: np.ndarray,
194     model: int,
195     radius_at: Callable[[float], float],
196     ambient_at: Callable[[float], tuple[float, float]],
197 ) -> Callable[[float, np.ndarray], np.ndarray]:
198     """Build the method-of-lines system for a fixed or shrinking cylinder."""
199     weights = storage_weights(x)
200     n = x.size
201
202     def rhs(time_s: float, state: np.ndarray) -> np.ndarray:
203         temperature = state[:n]
204         moisture = state[n:]
205         radius = float(radius_at(time_s))
206         ambient_t, ambient_c = ambient_at(time_s)
207         rho, cp, k, diffusivity = material_properties(
208             moisture, temperature, model
209         )
210         heat = transport_balance(
211             temperature, k, x, radius, HEAT_TRANSFER, ambient_t
212         )
213         mass = transport_balance(
214             moisture, diffusivity, x, radius, MASS_TRANSFER, ambient_c
215         )
216         scale = radius**2 * weights
217         return np.concatenate((heat / (scale * rho * cp), mass / scale))
218
219     return rhs
220
221
222 def solve_adaptive_bdf(
223     intervals: int,
224     end_time_s: float,
225     model: int,
226     radius_at: Callable[[float], float],
227     ambient_at: Callable[[float], tuple[float, float]],
228     rtol: float = 1.0e-10,
229     atol_t: float = 1.0e-10,
230     atol_c: float = 1.0e-12,
231     max_step_s: float = 2.0,
232 ) -> tuple[object, float | None]:
233     """Adaptive BDF driver used by Q2/Q4; locate the drying event."""
234     x = refined_grid(intervals)
235     n = x.size
236     initial = np.r_[np.full(n, INITIAL_T), np.full(n, INITIAL_C)]
237     solution = solve_ivp(
238         build_rhs(x, model, radius_at, ambient_at),
239         (0.0, end_time_s),
240         initial,
241         method="BDF",
242         rtol=rtol,
243         atol=np.r_[np.full(n, atol_t), np.full(n, atol_c)],
244         dense_output=True,
245         max_step=max_step_s,
246     )

```

```

247     if not solution.success:
248         raise RuntimeError(solution.message)
249
250     maximum_c = np.max(solution.y[n:], axis=0)
251     crossing = np.flatnonzero(
252         (maximum_c[:-1] > DRY_LIMIT) & (maximum_c[1:] <= DRY_LIMIT)
253     )
254     if crossing.size == 0:
255         return solution, None
256     j = int(crossing[0])
257     event = lambda t: float(np.max(solution.sol(t)[n:]) - DRY_LIMIT)
258     dry_time = brentq(event, solution.t[j], solution.t[j + 1])
259     return solution, float(dry_time)
260
261
262 def richardson_estimate(values: np.ndarray, safety: float = 1.5) -> tuple[float, float]:
263     """Observed order and fine-grid remainder for three 2:1 refinements."""
264     coarse, medium, fine = np.asarray(values, dtype=float)
265     order = np.log2(abs((coarse - medium) / (medium - fine)))
266     error = safety * abs(medium - fine) / (2.0**order - 1.0)
267     return float(order), float(error)
268
269
270 def dry_mass_ratio(
271     x: np.ndarray,
272     moisture: np.ndarray,
273     radius: float,
274     initial_radius: float = 0.02,
275 ) -> float:
276     """Question 4 posterior dry-solid conservation diagnostic."""
277     weights = storage_weights(x)
278     dry_density = (760.0 + 90.0 * moisture) / (1.0 + moisture)
279     initial_dry_density = (760.0 + 90.0 * INITIAL_C) / (1.0 + INITIAL_C)
280     current = radius**2 * np.sum(weights * dry_density)
281     initial = initial_radius**2 * 0.5 * initial_dry_density
282     return float(current / initial)
283
284
285 # Question 1 solver excerpt from the complete submitted program
286 def solve_tridiagonal(
287     lower: Sequence[float],
288     diagonal: Sequence[float],
289     upper: Sequence[float],
290     rhs: Sequence[float],
291 ) -> list[float]:
292     """Thomas algorithm with pivot checks for a tridiagonal linear system."""
293     n = len(diagonal)
294     if n == 0 or len(rhs) != n or len(lower) != n - 1 or len(upper) != n - 1:
295         raise ValueError("invalid tridiagonal dimensions")
296
297     modified_upper = [0.0] * (n - 1)
298     modified_rhs = [0.0] * n
299     pivot = diagonal[0]
300     if not math.isfinite(pivot) or abs(pivot) < 1.0e-300:
301         raise ArithmeticError("zero or non-finite pivot at row 0")
302     if n > 1:
303         modified_upper[0] = upper[0] / pivot
304     modified_rhs[0] = rhs[0] / pivot
305
306     for i in range(1, n):
307         pivot = diagonal[i] - lower[i - 1] * modified_upper[i - 1]
308         if not math.isfinite(pivot) or abs(pivot) < 1.0e-300:
309             raise ArithmeticError(f"zero or non-finite pivot at row {i}")
310         if i < n - 1:

```

```

311         modified_upper[i] = upper[i] / pivot
312         modified_rhs[i] = (rhs[i] - lower[i - 1] * modified_rhs[i - 1]) / pivot
313
314     solution = [0.0] * n
315     solution[-1] = modified_rhs[-1]
316     for i in range(n - 2, -1, -1):
317         solution[i] = modified_rhs[i] - modified_upper[i] * solution[i + 1]
318     return solution
319
320
321 def harmonic_mean(a: float, b: float) -> float:
322     denominator = a + b
323     if a <= 0.0 or b <= 0.0 or denominator <= 0.0:
324         raise ValueError("diffusion coefficients must be positive")
325     return 2.0 * a * b / denominator
326
327
328 def make_radial_grid(
329     radius_m: float,
330     radial_cells: int,
331     surface_layer_m: float,
332     surface_refinement_factor: int,
333 ) -> list[float]:
334     """Create a two-zone grid refined near the convective surface.
335
336     The bulk cell width is ``surface_refinement_factor`` times the surface
337     cell width. Requested 0.1 cm output radii are nodes for all default
338     comparison grids.
339     """
340     bulk_length = radius_m - surface_layer_m
341     effective_length = bulk_length + surface_refinement_factor * surface_layer_m
342     bulk_step = effective_length / radial_cells
343     surface_step = bulk_step / surface_refinement_factor
344     bulk_cells_float = bulk_length / bulk_step
345     surface_cells_float = surface_layer_m / surface_step
346     bulk_cells = int(round(bulk_cells_float))
347     surface_cells = int(round(surface_cells_float))
348     if (
349         abs(bulk_cells_float - bulk_cells) > 1.0e-10
350         or abs(surface_cells_float - surface_cells) > 1.0e-10
351         or bulk_cells + surface_cells != radial_cells
352     ):
353         raise ValueError(
354             "radial_cells is incompatible with the two-zone grid; "
355             "use a multiple of 19 for the default 2 mm / factor-10 refinement"
356         )
357     grid = [i * bulk_step for i in range(bulk_cells + 1)]
358     interface = bulk_length
359     grid.extend(
360         interface + i * surface_step for i in range(1, surface_cells + 1)
361     )
362     grid[0] = 0.0
363     grid[-1] = radius_m
364     if any(b <= a for a, b in zip(grid, grid[1:])):
365         raise AssertionError("constructed radial grid is not strictly increasing")
366     return grid
367
368
369 def make_time_coefficients(
370     old: Sequence[float], previous: Sequence[float] | None, dt: float
371 ) -> tuple[float, list[float]]:
372     if previous is None:
373         return 1.0 / dt, [value / dt for value in old]
374     factor = 1.0 / (2.0 * dt)

```

```

375     return 3.0 * factor, [
376         (4.0 * current - prior) * factor
377         for current, prior in zip(old, previous)
378     ]
379
380
381 def implicit_fvm_step(
382     old: Sequence[float],
383     previous: Sequence[float] | None,
384     dt: float,
385     radii_m: Sequence[float],
386     storage: float,
387     gamma_nodes: Sequence[float],
388     boundary_transfer: float,
389     external_value: float,
390 ) -> list[float]:
391     """One BE/BDF2 step for the conservative radial diffusion equation."""
392     node_count = len(old)
393     if (
394         node_count < 2
395         or len(gamma_nodes) != node_count
396         or len(radii_m) != node_count
397     ):
398         raise ValueError("inconsistent spatial arrays")
399     radial_cells = node_count - 1
400     radius_m = radii_m[-1]
401     if radii_m[0] != 0.0 or any(b <= a for a, b in zip(radii_m, radii_m[1:])):
402         raise ValueError("invalid radial grid")
403     time_diagonal, history = make_time_coefficients(old, previous, dt)
404
405     lower = [0.0] * radial_cells
406     diagonal = [0.0] * node_count
407     upper = [0.0] * radial_cells
408     rhs = [0.0] * node_count
409
410     gamma_east = harmonic_mean(gamma_nodes[0], gamma_nodes[1])
411     r_east = 0.5 * (radii_m[0] + radii_m[1])
412     east_conductance = r_east * gamma_east / (radii_m[1] - radii_m[0])
413     center_weight = r_east * r_east / 2.0
414     diagonal[0] = storage * center_weight * time_diagonal + east_conductance
415     upper[0] = -east_conductance
416     rhs[0] = storage * center_weight * history[0]
417
418     for i in range(1, radial_cells):
419         r_i = radii_m[i]
420         r_west = 0.5 * (radii_m[i - 1] + r_i)
421         r_east = 0.5 * (r_i + radii_m[i + 1])
422         gamma_west = harmonic_mean(gamma_nodes[i - 1], gamma_nodes[i])
423         gamma_east = harmonic_mean(gamma_nodes[i], gamma_nodes[i + 1])
424         west_conductance = r_west * gamma_west / (r_i - radii_m[i - 1])
425         east_conductance = r_east * gamma_east / (radii_m[i + 1] - r_i)
426         volume_weight = (r_east * r_east - r_west * r_west) / 2.0
427         lower[i - 1] = -west_conductance
428         diagonal[i] = (
429             storage * volume_weight * time_diagonal
430             + west_conductance
431             + east_conductance
432         )
433         upper[i] = -east_conductance
434         rhs[i] = storage * volume_weight * history[i]
435
436     r_west = 0.5 * (radii_m[-2] + radius_m)
437     gamma_west = harmonic_mean(gamma_nodes[-2], gamma_nodes[-1])
438     west_conductance = r_west * gamma_west / (radius_m - radii_m[-2])

```

```

439     boundary_conductance = radius_m * boundary_transfer
440     surface_weight = (radius_m * radius_m - r_west * r_west) / 2.0
441     lower[-1] = -west_conductance
442     diagonal[-1] = (
443         storage * surface_weight * time_diagonal
444         + west_conductance
445         + boundary_conductance
446     )
447     rhs[-1] = (
448         storage * surface_weight * history[-1]
449         + boundary_conductance * external_value
450     )
451
452     return solve_tridiagonal(lower, diagonal, upper, rhs)
453
454
455 def radial_storage_weights(radii_m: Sequence[float]) -> list[float]:
456     radial_cells = len(radii_m) - 1
457     radius_m = radii_m[-1]
458     weights = [0.0] * (radial_cells + 1)
459     first_face = 0.5 * (radii_m[0] + radii_m[1])
460     weights[0] = first_face * first_face / 2.0
461     for i in range(1, radial_cells):
462         west = 0.5 * (radii_m[i - 1] + radii_m[i])
463         east = 0.5 * (radii_m[i] + radii_m[i + 1])
464         weights[i] = (east * east - west * west) / 2.0
465     last_face = 0.5 * (radii_m[-2] + radius_m)
466     weights[-1] = (radius_m * radius_m - last_face * last_face) / 2.0
467     return weights
468
469
470 def update_balance_diagnostic(
471     diagnostics: SolverDiagnostics,
472     field_name: str,
473     new: Sequence[float],
474     old: Sequence[float],
475     previous: Sequence[float] | None,
476     dt: float,
477     weights: Sequence[float],
478     storage: float,
479     radius_m: float,
480     boundary_transfer: float,
481     external_value: float,
482 ) -> None:
483     time_diagonal, history = make_time_coefficients(old, previous, dt)
484     storage_rate = sum(
485         storage * weight * (time_diagonal * value - hist)
486         for weight, value, hist in zip(weights, new, history)
487     )
488     boundary_rate = radius_m * boundary_transfer * (external_value - new[-1])
489     residual = abs(storage_rate - boundary_rate)
490     scale = max(abs(storage_rate), abs(boundary_rate), 1.0e-30)
491     relative = residual / scale
492     if field_name == "heat":
493         diagnostics.max_heat_balance_abs = max(
494             diagnostics.max_heat_balance_abs, residual
495         )
496         diagnostics.max_heat_balance_rel = max(
497             diagnostics.max_heat_balance_rel, relative
498         )
499     elif field_name == "moisture":
500         diagnostics.max_moisture_balance_abs = max(
501             diagnostics.max_moisture_balance_abs, residual
502         )

```

```

503         diagnostics.max_moisture_balance_rel = max(
504             diagnostics.max_moisture_balance_rel, relative
505         )
506     else:
507         raise ValueError(field_name)
508
509
510 def solve_model(
511     environment: EnvironmentData,
512     parameters: PhysicalParameters,
513     config: NumericalConfig,
514     progress: bool = False,
515 ) -> SimulationResult:
516     config.validate()
517     environment.validate(config.end_time_s)
518     started = time.perf_counter()
519
520     n = config.radial_cells
521     dt = config.time_step_s
522     step_count = int(round(config.end_time_s / dt))
523     output_stride = int(round(config.output_interval_s / dt))
524     radii_m = make_radial_grid(
525         parameters.radius_m,
526         n,
527         config.surface_layer_m,
528         config.surface_refinement_factor,
529     )
530     radii_cm = [0.1 * i for i in range(21)]
531     requested_indices = []
532     for radius_cm in radii_cm:
533         target = radius_cm / 100.0
534         index = bisect.bisect_left(radii_m, target)
535         candidates = [j for j in (index - 1, index) if 0 <= j < len(radii_m)]
536         best = min(candidates, key=lambda j: abs(radii_m[j] - target))
537         if abs(radii_m[best] - target) > 1.0e-12:
538             raise ValueError(f"output radius {radius_cm} cm is not a grid node")
539         requested_indices.append(best)
540     weights = radial_storage_weights(radii_m)
541
542     temperature = [parameters.initial_temperature_c] * (n + 1)
543     moisture = [parameters.initial_moisture_kg_kg] * (n + 1)
544     previous_temperature: list[float] | None = None
545     previous_moisture: list[float] | None = None
546     constant_k = [parameters.conductivity_w_m_k] * (n + 1)
547
548     output_times: list[int] = []
549     temperature_output: list[list[float]] = []
550     moisture_output: list[list[float]] = []
551     diagnostics = SolverDiagnostics()
552
553     for step in range(1, step_count + 1):
554         t_new = step * dt
555         external_temperature, external_moisture = environment.interpolate(t_new)
556
557         new_temperature = implicit_fvm_step(
558             temperature,
559             previous_temperature,
560             dt,
561             radii_m,
562             parameters.volumetric_heat_capacity,
563             constant_k,
564             parameters.heat_transfer_w_m2_k,
565             external_temperature,
566         )

```

```

567
568 guess = moisture.copy()
569 final_update = math.inf
570 for iteration in range(1, config.picard_max_iterations + 1):
571     diffusivity = [parameters.moisture_diffusivity(value) for value in guess]
572     candidate = implicit_fvm_step(
573         moisture,
574         previous_moisture,
575         dt,
576         radii_m,
577         1.0,
578         diffusivity,
579         parameters.mass_transfer_m_s,
580         external_moisture,
581     )
582     final_update = max(abs(a - b) for a, b in zip(candidate, guess))
583     guess = candidate
584     if final_update < config.picard_tolerance:
585         break
586     else:
587         raise RuntimeError(
588             f"Picard iteration failed at t={t_new:.9g} s; "
589             f"last update={final_update:.3e}"
590         )
591     new_moisture = guess
592
593     diagnostics.moisture_steps += 1
594     diagnostics.total_picard_iterations += iteration
595     diagnostics.max_picard_iterations = max(
596         diagnostics.max_picard_iterations, iteration
597     )
598     diagnostics.max_picard_update = max(
599         diagnostics.max_picard_update, final_update
600     )
601
602     update_balance_diagnostic(
603         diagnostics,
604         "heat",
605         new_temperature,
606         temperature,
607         previous_temperature,
608         dt,
609         weights,
610         parameters.volumetric_heat_capacity,
611         parameters.radius_m,
612         parameters.heat_transfer_w_m2_k,
613         external_temperature,
614     )
615     update_balance_diagnostic(
616         diagnostics,
617         "moisture",
618         new_moisture,
619         moisture,
620         previous_moisture,
621         dt,
622         weights,
623         1.0,
624         parameters.radius_m,
625         parameters.mass_transfer_m_s,
626         external_moisture,
627     )
628
629     previous_temperature, temperature = temperature, new_temperature
630     previous_moisture, moisture = moisture, new_moisture

```



```

631
632         diagnostics.min_temperature_c = min(
633             diagnostics.min_temperature_c, min(temperature)
634         )
635         diagnostics.max_temperature_c = max(
636             diagnostics.max_temperature_c, max(temperature)
637         )
638         diagnostics.min_moisture_kg_kg = min(
639             diagnostics.min_moisture_kg_kg, min(moisture)
640         )
641         diagnostics.max_moisture_kg_kg = max(
642             diagnostics.max_moisture_kg_kg, max(moisture)
643         )
644
645         if step % output_stride == 0:
646             output_times.append(int(round(t_new)))
647             temperature_output.append([temperature[i] for i in requested_indices])
648             moisture_output.append([moisture[i] for i in requested_indices])
649
650         if progress and (step % max(1, step_count // 10) == 0):
651             print(
652                 f" {100.0 * step / step_count:5.1f}% "
653                 f"t={t_new:8.3f} s Picard={iteration}",
654                 flush=True,
655             )
656
657         if output_times != list(range(1, int(config.end_time_s) + 1)):
658             raise AssertionError("unexpected output time grid")
659         return SimulationResult(
660             times_s=output_times,
661             radii_cm=radii_cm,
662             temperatures_c=temperature_output,
663             moistures_kg_kg=moisture_output,
664             diagnostics=diagnostics,
665             runtime_s=time.perf_counter() - started,
666         )
667
668

```

B 问题二完整程序的关键实现

以下摘录给出表面加密网格、控制体权重、Jacobian 稀疏结构、变物性耦合方程、自适应 BDF 求解及空间收敛计算。

```

1  # Question 2 core routines
2  def make_radial_grid(
3      radius_m: float,
4      radial_cells: int,
5      surface_layer_m: float,
6      surface_refinement_factor: int,
7  ) -> np.ndarray:
8      """Construct a two-zone grid refined in the outermost surface layer."""
9      bulk_length = radius_m - surface_layer_m
10     effective_length = bulk_length + surface_refinement_factor * surface_layer_m
11     bulk_step = effective_length / radial_cells
12     surface_step = bulk_step / surface_refinement_factor
13     bulk_cells_float = bulk_length / bulk_step
14     surface_cells_float = surface_layer_m / surface_step
15     bulk_cells = int(round(bulk_cells_float))

```

```

16     surface_cells = int(round(surface_cells_float))
17     if (
18         abs(bulk_cells_float - bulk_cells) > 1.0e-10
19         or abs(surface_cells_float - surface_cells) > 1.0e-10
20         or bulk_cells + surface_cells != radial_cells
21     ):
22         raise ValueError(
23             "radial_cells is incompatible with the two-zone grid; "
24             "use a multiple of 19 for the 2 mm / factor-10 refinement"
25         )
26     bulk = np.arange(bulk_cells + 1, dtype=float) * bulk_step
27     surface = bulk_length + np.arange(1, surface_cells + 1, dtype=float) * surface_step
28     radii = np.concatenate((bulk, surface))
29     radii[0] = 0.0
30     radii[-1] = radius_m
31     if radii.size != radial_cells + 1 or np.any(np.diff(radii) <= 0.0):
32         raise AssertionError("constructed radial grid is invalid")
33     return radii
34
35
36 def radial_storage_weights(radii_m: np.ndarray) -> np.ndarray:
37     faces = 0.5 * (radii_m[:-1] + radii_m[1:])
38     weights = np.empty_like(radii_m)
39     weights[0] = 0.5 * faces[0] ** 2
40     weights[1:-1] = 0.5 * (faces[1:] ** 2 - faces[:-1] ** 2)
41     weights[-1] = 0.5 * (radii_m[-1] ** 2 - faces[-1] ** 2)
42     return weights
43
44
45 def harmonic_mean(left: np.ndarray, right: np.ndarray) -> np.ndarray:
46     denominator = left + right
47     if np.any(left <= 0.0) or np.any(right <= 0.0) or np.any(denominator <= 0.0):
48         raise ValueError("interface transport coefficients must be positive")
49     return 2.0 * left * right / denominator
50
51
52 def make_jacobian_sparsity(node_count: int) -> csr_matrix:
53     """Return the block-tridiagonal dependency pattern of the coupled system."""
54     pattern = lil_matrix((2 * node_count, 2 * node_count), dtype=np.int8)
55     for i in range(node_count):
56         neighbours = range(max(0, i - 1), min(node_count, i + 2))
57         for j in neighbours:
58             pattern[i, j] = 1
59             pattern[i, node_count + j] = 1
60             pattern[node_count + i, j] = 1
61             pattern[node_count + i, node_count + j] = 1
62     return pattern.tocsr()
63
64
65 class CoupledRadialSystem:
66     def __init__(
67         self,
68         environment: EnvironmentData,
69         parameters: PhysicalParameters,
70         radii_m: np.ndarray,
71     ) -> None:
72         self.environment = environment
73         self.parameters = parameters
74         self.radii_m = radii_m
75         self.node_count = radii_m.size
76         self.radius_m = float(radii_m[-1])
77         self.faces_m = 0.5 * (radii_m[:-1] + radii_m[1:])
78         self.spacings_m = np.diff(radii_m)
79         self.weights_m2 = radial_storage_weights(radii_m)

```

```

80
81 def transport_net(
82     self,
83     field: np.ndarray,
84     gamma_nodes: np.ndarray,
85     boundary_transfer: float,
86     external_value: float,
87 ) -> np.ndarray:
88     gamma_faces = harmonic_mean(gamma_nodes[:-1], gamma_nodes[1:])
89     interface_flux = (
90         self.faces_m
91         * gamma_faces
92         * np.diff(field)
93         / self.spacing_m
94     )
95     net = np.empty_like(field)
96     net[0] = interface_flux[0]
97     net[1:-1] = interface_flux[1:] - interface_flux[:-1]
98     net[-1] = (
99         self.radius_m * boundary_transfer * (external_value - field[-1])
100         - interface_flux[-1]
101     )
102     return net
103
104 def __call__(self, t_s: float, state: np.ndarray) -> np.ndarray:
105     n = self.node_count
106     temperature_c = state[:n]
107     moisture = state[n:]
108     density, heat_capacity, conductivity, diffusivity = (
109         self.parameters.evaluate_properties(moisture, temperature_c)
110     )
111     external_temperature, external_moisture = self.environment.interpolate(t_s)
112     heat_net = self.transport_net(
113         temperature_c,
114         conductivity,
115         self.parameters.heat_transfer_w_m2_k,
116         external_temperature,
117     )
118     moisture_net = self.transport_net(
119         moisture,
120         diffusivity,
121         self.parameters.mass_transfer_m_s,
122         external_moisture,
123     )
124     temperature_rate = heat_net / (
125         self.weights_m2 * density * heat_capacity
126     )
127     moisture_rate = moisture_net / self.weights_m2
128     return np.concatenate((temperature_rate, moisture_rate))
129
130
131 def requested_output_indices(radii_m: np.ndarray) -> tuple[np.ndarray, np.ndarray]:
132     requested_cm = np.arange(21, dtype=float) * 0.1
133     indices = np.empty(requested_cm.size, dtype=int)
134     for j, radius_cm in enumerate(requested_cm):
135         target_m = radius_cm / 100.0
136         index = int(np.argmax(np.abs(radii_m - target_m)))
137         if abs(float(radii_m[index]) - target_m) > 1.0e-12:
138             raise ValueError(f"requested radius {radius_cm:g} cm is not a grid node")
139         indices[j] = index
140     return requested_cm, indices
141
142
143 def moisture_balance_diagnostic(

```

```

144     environment: EnvironmentData,
145     parameters: PhysicalParameters,
146     weights: np.ndarray,
147     times_s: np.ndarray,
148     all_moistures: np.ndarray,
149 ) -> tuple[float, float]:
150     inventory = weights @ all_moistures
151     surface = all_moistures[-1, :]
152     external = np.interp(times_s, environment.times_s, environment.moistures_kg_kg)
153     boundary_rate = (
154         parameters.radius_m
155         * parameters.mass_transfer_m_s
156         * (external - surface)
157     )
158     dt = np.diff(times_s)
159     trapezoidal_change = 0.5 * (boundary_rate[:-1] + boundary_rate[1:]) * dt
160     residual = np.diff(inventory) - trapezoidal_change
161     scale = np.maximum.reduce(
162         (
163             np.abs(np.diff(inventory)),
164             np.abs(trapezoidal_change),
165             np.full_like(residual, 1.0e-30),
166         )
167     )
168     return float(np.max(np.abs(residual))), float(np.max(np.abs(residual) / scale))
169
170
171 def solve_model(
172     environment: EnvironmentData,
173     parameters: PhysicalParameters,
174     config: NumericalConfig,
175     progress_label: str | None = None,
176     integration_method: str = "BDF",
177 ) -> SimulationResult:
178     if integration_method not in {"BDF", "Radau"}:
179         raise ValueError("integration_method must be 'BDF' or 'Radau'")
180     config.validate()
181     environment.validate(config.end_time_s)
182     radii_m = make_radial_grid(
183         parameters.radius_m,
184         config.radial_cells,
185         config.surface_layer_m,
186         config.surface_refinement_factor,
187     )
188     system = CoupledRadialSystem(environment, parameters, radii_m)
189     node_count = radii_m.size
190     state0 = np.concatenate(
191         (
192             np.full(node_count, parameters.initial_temperature_c, dtype=float),
193             np.full(node_count, parameters.initial_moisture_kg_kg, dtype=float),
194         )
195     )
196     output_times = np.arange(
197         0.0,
198         config.end_time_s + 0.5 * config.output_interval_s,
199         config.output_interval_s,
200         dtype=float,
201     )
202     absolute_tolerance = np.concatenate(
203         (
204             np.full(node_count, config.temperature_absolute_tolerance),
205             np.full(node_count, config.moisture_absolute_tolerance),
206         )
207     )

```

```

208     if progress_label:
209         print(
210             f"[{progress_label}] N={config.radial_cells}, "
211             f"rtol={config.relative_tolerance:.1e}, "
212             f"max_step={config.maximum_step_s:g} s",
213             flush=True,
214         )
215     started = time.perf_counter()
216     solution = solve_ivp(
217         system,
218         (0.0, config.end_time_s),
219         state0,
220         method=integration_method,
221         t_eval=output_times,
222         rtol=config.relative_tolerance,
223         atol=absolute_tolerance,
224         jac_sparsity=make_jacobian_sparsity(node_count),
225         max_step=config.maximum_step_s,
226         first_step=min(config.first_step_s, config.end_time_s),
227     )
228     runtime_s = time.perf_counter() - started
229     if not solution.success:
230         raise RuntimeError(
231             f"{integration_method} integration failed: {solution.message}"
232         )
233     if solution.t.size != output_times.size or not np.array_equal(
234         solution.t, output_times
235     ):
236         raise RuntimeError(
237             f"{integration_method} solver did not return the requested output time grid"
238         )
239     if not np.all(np.isfinite(solution.y)):
240         raise RuntimeError(f"{integration_method} solution contains non-finite values")
241
242     all_temperature = solution.y[:node_count, :]
243     all_moisture = solution.y[node_count:, :]
244     if np.min(all_moisture) <= 0.0:
245         raise RuntimeError("computed a non-positive moisture value")
246
247     density, heat_capacity, conductivity, diffusivity = parameters.evaluate_properties(
248         all_moisture.ravel(), all_temperature.ravel()
249     )
250     balance_abs, balance_rel = moisture_balance_diagnostics(
251         environment,
252         parameters,
253         system.weights_m2,
254         output_times,
255         all_moisture,
256     )
257     radii_cm, indices = requested_output_indices(radii_m)
258     result = SimulationResult(
259         times_s=output_times[1:].astype(int),
260         radii_cm=radii_cm,
261         temperatures_c=all_temperature[indices, 1:].T.copy(),
262         moistures_kg_kg=all_moisture[indices, 1:].T.copy(),
263         diagnostics=SolverDiagnostics(
264             runtime_s=runtime_s,
265             success=bool(solution.success),
266             message=str(solution.message),
267             function_evaluations=int(solution.nfev),
268             jacobian_evaluations=int(solution.njev),
269             lu_decompositions=int(solution.nlu),
270             returned_time_points=int(solution.t.size),
271             min_temperature_c=float(np.min(all_temperature)),

```

```

272         max_temperature_c=float(np.max(all_temperature)),
273         min_moisture_kg_kg=float(np.min(all_moisture)),
274         max_moisture_kg_kg=float(np.max(all_moisture)),
275         min_density_kg_m3=float(np.min(density)),
276         max_density_kg_m3=float(np.max(density)),
277         min_heat_capacity_j_kg_k=float(np.min(heat_capacity)),
278         max_heat_capacity_j_kg_k=float(np.max(heat_capacity)),
279         min_conductivity_w_m_k=float(np.min(conductivity)),
280         max_conductivity_w_m_k=float(np.max(conductivity)),
281         min_diffusivity_m2_s=float(np.min(diffusivity)),
282         max_diffusivity_m2_s=float(np.max(diffusivity)),
283         max_one_second_moisture_balance_abs=balance_abs,
284         max_one_second_moisture_balance_rel=balance_rel,
285     ),
286     config=config,
287 )
288 if progress_label:
289     print(
290         f"[{progress_label}] complete in {runtime_s:.2f} s; "
291         f"nfev={solution.nfev}, nlu={solution.nlu}",
292         flush=True,
293     )
294 return result
295
296
297

```

C 问题三完整程序的关键实现

以下摘录给出后向 Euler 启动、BDF2 时间推进、Picard 耦合迭代、全域阈值事件判断及临界时刻定位。

```

1  # Question 3 core routines
2  def time_coefficients(
3      old: Sequence[float], previous: Sequence[float] | None, dt_s: float
4  ) -> tuple[float, list[float]]:
5      """Return BE coefficients for step one and BDF2 thereafter."""
6      if previous is None:
7          return 1.0 / dt_s, [value / dt_s for value in old]
8      factor = 1.0 / (2.0 * dt_s)
9      return 3.0 * factor, [
10         (4.0 * current - prior) * factor
11         for current, prior in zip(old, previous)
12     ]
13
14
15  def implicit_radial_step(
16      old: Sequence[float],
17      previous: Sequence[float] | None,
18      dt_s: float,
19      radii_m: Sequence[float],
20      storage_nodes: Sequence[float],
21      gamma_faces: Sequence[float],
22      boundary_transfer: float,
23      external_value: float,
24  ) -> list[float]:
25      """One conservative BE/BDF2 radial finite-volume step."""
26      node_count = len(old)
27      radial_cells = node_count - 1

```

```

28     if node_count < 2:
29         raise ValueError("at least two radial nodes are required")
30     if previous is not None and len(previous) != node_count:
31         raise ValueError("invalid history arrays")
32     if len(radial_m) != node_count or len(storage_nodes) != node_count:
33         raise ValueError("invalid nodal arrays")
34     if len(gamma_faces) != radial_cells:
35         raise ValueError("invalid face coefficient array")
36     if any(value <= 0.0 or not math.isfinite(value) for value in storage_nodes):
37         raise ValueError("storage coefficients must be positive")
38     if any(value <= 0.0 or not math.isfinite(value) for value in gamma_faces):
39         raise ValueError("face coefficients must be positive")
40
41     time_diagonal, history = time_coefficients(old, previous, dt_s)
42     lower = [0.0] * radial_cells
43     diagonal = [0.0] * node_count
44     upper = [0.0] * radial_cells
45     rhs = [0.0] * node_count
46
47     east = 0.5 * (radial_m[0] + radial_m[1])
48     east_conductance = east * gamma_faces[0] / (radial_m[1] - radial_m[0])
49     centre_weight = east**2 / 2.0
50     diagonal[0] = storage_nodes[0] * centre_weight * time_diagonal + east_conductance
51     upper[0] = -east_conductance
52     rhs[0] = storage_nodes[0] * centre_weight * history[0]
53
54     for i in range(1, radial_cells):
55         west = 0.5 * (radial_m[i - 1] + radial_m[i])
56         east = 0.5 * (radial_m[i] + radial_m[i + 1])
57         west_conductance = west * gamma_faces[i - 1] / (radial_m[i] - radial_m[i - 1])
58         east_conductance = east * gamma_faces[i] / (radial_m[i + 1] - radial_m[i])
59         weight = (east**2 - west**2) / 2.0
60         lower[i - 1] = -west_conductance
61         diagonal[i] = (
62             storage_nodes[i] * weight * time_diagonal
63             + west_conductance
64             + east_conductance
65         )
66         upper[i] = -east_conductance
67         rhs[i] = storage_nodes[i] * weight * history[i]
68
69     radius = radial_m[-1]
70     west = 0.5 * (radial_m[-2] + radius)
71     west_conductance = west * gamma_faces[-1] / (radius - radial_m[-2])
72     boundary_conductance = radius * boundary_transfer
73     surface_weight = (radius**2 - west**2) / 2.0
74     lower[-1] = -west_conductance
75     diagonal[-1] = (
76         storage_nodes[-1] * surface_weight * time_diagonal
77         + west_conductance
78         + boundary_conductance
79     )
80     rhs[-1] = (
81         storage_nodes[-1] * surface_weight * history[-1]
82         + boundary_conductance * external_value
83     )
84     return solve_tridiagonal(lower, diagonal, upper, rhs)
85
86
87 def conductivity_faces(
88     moisture: Sequence[float], parameters: PhysicalParameters
89 ) -> list[float]:
90     return [
91         parameters.conductivity(0.5 * (left + right))

```

```

92         for left, right in zip(moisture, moisture[1:])
93     ]
94
95
96 def integrated_face_diffusivity(
97     left_moisture: float,
98     right_moisture: float,
99     face_temperature_k: float,
100     parameters: PhysicalParameters,
101 ) -> float:
102     """Kirchhoff/secant mean of D across one moisture face."""
103     midpoint = 0.5 * (left_moisture + right_moisture)
104     half_span = 0.5 * (right_moisture - left_moisture)
105     # The temperature-dependent factor is constant across this face integral;
106     # evaluate it once instead of four times. This matters for the N=160,
107     # dt=5 s production experiment, which evaluates millions of face fluxes.
108     thermal_factor = 2.4e-3 * math.exp(-3850.0 / face_temperature_k)
109     total = 0.0
110     for point, weight in zip(GAUSS_X, GAUSS_W):
111         moisture = midpoint + half_span * point
112         parameters._check_moisture(moisture)
113         total += weight * math.exp(-0.45 / moisture)
114     return 0.5 * thermal_factor * total
115
116
117 def diffusivity_faces(
118     moisture: Sequence[float],
119     temperature_c: Sequence[float],
120     parameters: PhysicalParameters,
121 ) -> list[float]:
122     result = []
123     for c_left, c_right, t_left, t_right in zip(
124         moisture, moisture[1:], temperature_c, temperature_c[1:]
125     ):
126         face_temperature_k = 0.5 * (t_left + t_right) + 273.15
127         result.append(
128             integrated_face_diffusivity(
129                 c_left, c_right, face_temperature_k, parameters
130             )
131         )
132     return result
133
134
135 def requested_node_indices(radial_cells: int) -> tuple[list[float], list[int]]:
136     radii_cm = [0.1 * i for i in range(21)]
137     stride = radial_cells // 20
138     return radii_cm, [i * stride for i in range(21)]
139
140
141 def interpolate_vector(
142     old: Sequence[float], new: Sequence[float], fraction: float
143 ) -> list[float]:
144     return [a + fraction * (b - a) for a, b in zip(old, new)]
145
146
147 def balance_residual(
148     new: Sequence[float],
149     old: Sequence[float],
150     previous: Sequence[float] | None,
151     dt_s: float,
152     weights: Sequence[float],
153     storage_nodes: Sequence[float],
154     radius_m: float,
155     boundary_transfer: float,

```



```

156     external_value: float,
157     relative_scale_floor: float = 1.0e-10,
158 ) -> tuple[float, float]:
159     time_diagonal, history = time_coefficients(old, previous, dt_s)
160     storage_rate = sum(
161         storage * weight * (time_diagonal * value - hist)
162         for storage, weight, value, hist in zip(
163             storage_nodes, weights, new, history
164         )
165     )
166     boundary_rate = radius_m * boundary_transfer * (external_value - new[-1])
167     absolute = abs(storage_rate - boundary_rate)
168     # A relative residual is not informative after the field has equilibrated:
169     # both sides then approach roundoff and their quotient can be 0(1) despite
170     # a tiny absolute residual. Retain the absolute residual in all cases and
171     # evaluate the relative metric only while the balance has a useful scale.
172     scale = max(abs(storage_rate), abs(boundary_rate))
173     relative = 0.0 if scale < relative_scale_floor else absolute / scale
174     return absolute, relative
175
176
177 def solve_model(
178     environment: EnvironmentData,
179     parameters: PhysicalParameters,
180     config: NumericalConfig,
181     progress: bool = False,
182 ) -> SimulationResult:
183     config.validate()
184     environment.validate()
185     started = time.perf_counter()
186     radii_m = make_uniform_grid(parameters.radius_m, config.radial_cells)
187     weights = radial_storage_weights(radii_m)
188     output_radii_cm, output_indices = requested_node_indices(config.radial_cells)
189     output_stride = int(round(config.output_interval_s / config.time_step_s))
190     maximum_steps = int(math.ceil(config.maximum_time_h * 3600.0 / config.time_step_s))
191
192     temperature = [parameters.initial_temperature_c] * (config.radial_cells + 1)
193     moisture = [parameters.initial_moisture_kg_kg] * (config.radial_cells + 1)
194     previous_temperature: list[float] | None = None
195     previous_moisture: list[float] | None = None
196     previous_gap = max(moisture) - parameters.moisture_threshold_kg_kg
197
198     output_times: list[float] = []
199     output_moistures: list[list[float]] = []
200     diagnostics = SolverDiagnostics()
201     next_progress_s = 6.0 * 3600.0
202
203     for step in range(1, maximum_steps + 1):
204         t_new = step * config.time_step_s
205         external_temperature, external_moisture = environment.value(t_new)
206         guess_temperature = temperature.copy()
207         guess_moisture = moisture.copy()
208
209         for iteration in range(1, config.picard_max_iterations + 1):
210             heat_storage = [
211                 parameters.density(value) * parameters.heat_capacity(value)
212                 for value in guess_moisture
213             ]
214             heat_faces = conductivity_faces(guess_moisture, parameters)
215             candidate_temperature = implicit_radial_step(
216                 temperature,
217                 previous_temperature,
218                 config.time_step_s,
219                 radii_m,

```

```

220         heat_storage,
221         heat_faces,
222         parameters.heat_transfer_w_m2_k,
223         external_temperature,
224     )
225     moisture_faces = diffusivity_faces(
226         guess_moisture, candidate_temperature, parameters
227     )
228     candidate_moisture = implicit_radial_step(
229         moisture,
230         previous_moisture,
231         config.time_step_s,
232         radii_m,
233         [1.0] * len(radii_m),
234         moisture_faces,
235         parameters.mass_transfer_m_s,
236         external_moisture,
237     )
238     temperature_update = max(
239         abs(a - b)
240         for a, b in zip(candidate_temperature, guess_temperature)
241     )
242     moisture_update = max(
243         abs(a - b) for a, b in zip(candidate_moisture, guess_moisture)
244     )
245     combined_update = max(temperature_update / 50.0, moisture_update)
246     guess_temperature = candidate_temperature
247     guess_moisture = candidate_moisture
248     if combined_update < config.picard_tolerance:
249         break
250     else:
251         raise RuntimeError(
252             f"Picard iteration failed at t={t_new:.3f} s; "
253             f"last scaled update={combined_update:.3e}"
254         )
255
256     new_temperature = guess_temperature
257     new_moisture = guess_moisture
258     final_heat_storage = [
259         parameters.density(value) * parameters.heat_capacity(value)
260         for value in new_moisture
261     ]
262     heat_abs, heat_rel = balance_residual(
263         new_temperature,
264         temperature,
265         previous_temperature,
266         config.time_step_s,
267         weights,
268         final_heat_storage,
269         parameters.radius_m,
270         parameters.heat_transfer_w_m2_k,
271         external_temperature,
272         relative_scale_floor=1.0e-8,
273     )
274     moisture_abs, moisture_rel = balance_residual(
275         new_moisture,
276         moisture,
277         previous_moisture,
278         config.time_step_s,
279         weights,
280         [1.0] * len(radii_m),
281         parameters.radius_m,
282         parameters.mass_transfer_m_s,
283         external_moisture,

```

```

284         relative_scale_floor=1.0e-14,
285     )
286
287     diagnostics.steps += 1
288     diagnostics.total_picard_iterations += iteration
289     diagnostics.max_picard_iterations = max(
290         diagnostics.max_picard_iterations, iteration
291     )
292     diagnostics.final_picard_update = combined_update
293     diagnostics.max_heat_balance_abs = max(diagnostics.max_heat_balance_abs, heat_abs)
294     diagnostics.max_heat_balance_rel = max(diagnostics.max_heat_balance_rel, heat_rel)
295     diagnostics.max_moisture_balance_abs = max(
296         diagnostics.max_moisture_balance_abs, moisture_abs
297     )
298     diagnostics.max_moisture_balance_rel = max(
299         diagnostics.max_moisture_balance_rel, moisture_rel
300     )
301     diagnostics.max_radial_monotonicity_violation = max(
302         diagnostics.max_radial_monotonicity_violation,
303         max(
304             (outer - inner for inner, outer in zip(new_moisture, new_moisture[1:])),
305             default=0.0,
306         ),
307     )
308     diagnostics.min_temperature_c = min(
309         diagnostics.min_temperature_c, min(new_temperature)
310     )
311     diagnostics.max_temperature_c = max(
312         diagnostics.max_temperature_c, max(new_temperature)
313     )
314     diagnostics.min_moisture_kg_kg = min(
315         diagnostics.min_moisture_kg_kg, min(new_moisture)
316     )
317     diagnostics.max_moisture_kg_kg = max(
318         diagnostics.max_moisture_kg_kg, max(new_moisture)
319     )
320
321     new_gap = max(new_moisture) - parameters.moisture_threshold_kg_kg
322     if new_gap < 0.0 <= previous_gap:
323         fraction = previous_gap / (previous_gap - new_gap)
324         fraction = min(1.0, max(0.0, fraction))
325         drying_time_s = t_new - config.time_step_s + fraction * config.time_step_s
326         drying_state = interpolate_vector(moisture, new_moisture, fraction)
327         requested_drying_state = [drying_state[i] for i in output_indices]
328         if not output_times or abs(output_times[-1] - drying_time_s) > 1.0e-9:
329             output_times.append(drying_time_s)
330             output_moistures.append(requested_drying_state)
331     return SimulationResult(
332         times_s=output_times,
333         radii_cm=output_radii_cm,
334         moistures_kg_kg=output_moistures,
335         drying_time_s=drying_time_s,
336         drying_moistures_kg_kg=requested_drying_state,
337         diagnostics=diagnostics,
338         runtime_s=time.perf_counter() - started,
339         config=config,
340     )
341
342     if step % output_stride == 0:
343         output_times.append(t_new)
344         output_moistures.append([new_moisture[i] for i in output_indices])
345
346     previous_temperature, temperature = temperature, new_temperature
347     previous_moisture, moisture = moisture, new_moisture

```

```

348     previous_gap = new_gap
349
350     if progress and t_new + 1.0e-9 >= next_progress_s:
351         print(
352             f" t={t_new / 3600.0:7.2f} h "
353             f"Cmax={max(moisture):.8f} Csurf={moisture[-1]:.8f} "
354             f"Picard={iteration}",
355             flush=True,
356         )
357         next_progress_s += 6.0 * 3600.0
358
359     raise RuntimeError(
360         f"drying criterion was not reached before {config.maximum_time_h:g} h; "
361         f"last maximum moisture={max(moisture):.9g}"
362     )
363
364

```

D 问题四完整程序的关键实现

以下摘录给出时变半径下的固定域变换、移动边界热湿方程、连续临界时刻求解、干质量后验检验及三级网格收敛分析。

```

1  # Question 4 core routines
2  def make_reference_grid(
3      radial_cells: int,
4      surface_layer: float = 0.1,
5      refinement_factor: int = 10,
6  ) -> np.ndarray:
7      bulk_length = 1.0 - surface_layer
8      effective_length = bulk_length + refinement_factor * surface_layer
9      bulk_step = effective_length / radial_cells
10     surface_step = bulk_step / refinement_factor
11     bulk_cells_float = bulk_length / bulk_step
12     surface_cells_float = surface_layer / surface_step
13     bulk_cells = int(round(bulk_cells_float))
14     surface_cells = int(round(surface_cells_float))
15     if (
16         abs(bulk_cells_float - bulk_cells) > 1.0e-10
17         or abs(surface_cells_float - surface_cells) > 1.0e-10
18         or bulk_cells + surface_cells != radial_cells
19     ):
20         raise ValueError("radial_cells is incompatible with the two-zone grid")
21     bulk = np.arange(bulk_cells + 1, dtype=float) * bulk_step
22     surface = bulk_length + np.arange(1, surface_cells + 1, dtype=float) * surface_step
23     xi = np.concatenate((bulk, surface))
24     xi[0] = 0.0
25     xi[-1] = 1.0
26     if xi.size != radial_cells + 1 or np.any(np.diff(xi) <= 0.0):
27         raise AssertionError("invalid reference grid")
28     return xi
29
30
31 def radial_storage_weights(xi: np.ndarray) -> np.ndarray:
32     faces = 0.5 * (xi[:-1] + xi[1:])
33     weights = np.empty_like(xi)
34     weights[0] = 0.5 * faces[0] ** 2
35     weights[1:-1] = 0.5 * (faces[1:] ** 2 - faces[:-1] ** 2)
36     weights[-1] = 0.5 * (1.0 - faces[-1] ** 2)

```

```

37     return weights
38
39
40 def harmonic_mean(left: np.ndarray, right: np.ndarray) -> np.ndarray:
41     if np.any(left <= 0.0) or np.any(right <= 0.0):
42         raise ValueError("transport coefficient is non-positive")
43     return 2.0 * left * right / (left + right)
44
45
46 def make_jacobian_sparsity(node_count: int) -> csr_matrix:
47     """Jacobian pattern for T, C and an auxiliary boundary-flux integral."""
48
49     state_size = 2 * node_count + 1
50     pattern = lil_matrix((state_size, state_size), dtype=np.int8)
51     for i in range(node_count):
52         for j in range(max(0, i - 1), min(node_count, i + 2)):
53             pattern[i, j] = 1
54             pattern[i, node_count + j] = 1
55             pattern[node_count + i, j] = 1
56             pattern[node_count + i, node_count + j] = 1
57     pattern[-1, 2 * node_count - 1] = 1
58     return pattern.tocsr()
59
60
61 class MovingBoundarySystem:
62     def __init__(
63         self,
64         environment: EnvironmentData,
65         radius_data: RadiusData,
66         parameters: PhysicalParameters,
67         xi: np.ndarray,
68     ) -> None:
69         self.environment = environment
70         self.radius_data = radius_data
71         self.parameters = parameters
72         self.xi = xi
73         self.node_count = xi.size
74         self.faces = 0.5 * (xi[:-1] + xi[1:])
75         self.spacing = np.diff(xi)
76         self.weights = radial_storage_weights(xi)
77
78     def transport_net(
79         self,
80         field: np.ndarray,
81         gamma: np.ndarray,
82         radius_m: float,
83         boundary_transfer: float,
84         external_value: float,
85     ) -> np.ndarray:
86         gamma_faces = harmonic_mean(gamma[:-1], gamma[1:])
87         internal_flux = self.faces * gamma_faces * np.diff(field) / self.spacing
88         net = np.empty_like(field)
89         net[0] = internal_flux[0]
90         net[1:-1] = internal_flux[1:] - internal_flux[:-1]
91         net[-1] = (
92             radius_m * boundary_transfer * (external_value - field[-1])
93             - internal_flux[-1]
94         )
95         return net
96
97     def __call__(self, t_s: float, state: np.ndarray) -> np.ndarray:
98         n = self.node_count
99         temperature = state[:n]
100         moisture = state[n : 2 * n]

```

```

101     density, heat_capacity, conductivity, diffusivity = (
102         self.parameters.evaluate_properties(moisture, temperature)
103     )
104     ambient_temperature, ambient_moisture = self.environment.interpolate(t_s)
105     radius_m = float(self.radius_data.interpolate(t_s))
106     heat_net = self.transport_net(
107         temperature,
108         conductivity,
109         radius_m,
110         self.parameters.heat_transfer_w_m2_k,
111         float(ambient_temperature),
112     )
113     moisture_net = self.transport_net(
114         moisture,
115         diffusivity,
116         radius_m,
117         self.parameters.mass_transfer_m_s,
118         float(ambient_moisture),
119     )
120     radius_squared = radius_m * radius_m
121     temperature_rate = heat_net / (
122         radius_squared * self.weights * density * heat_capacity
123     )
124     moisture_rate = moisture_net / (radius_squared * self.weights)
125     boundary_inventory_rate = (
126         self.parameters.mass_transfer_m_s
127         / radius_m
128         * (float(ambient_moisture) - moisture[-1])
129     )
130     return np.concatenate(
131         (temperature_rate, moisture_rate, np.asarray([boundary_inventory_rate]))
132     )
133
134
135 def interpolate_rows(
136     source_x: np.ndarray, rows: np.ndarray, target_x: np.ndarray
137 ) -> np.ndarray:
138     output = np.empty((rows.shape[0], target_x.size), dtype=float)
139     for i, row in enumerate(rows):
140         output[i] = np.interp(target_x, source_x, row)
141     return output
142
143
144 def extract_physical_moistures(
145     xi: np.ndarray,
146     moisture_rows: np.ndarray,
147     radii_m: np.ndarray,
148     fixed_radii_cm: np.ndarray,
149 ) -> np.ndarray:
150     output = np.full((moisture_rows.shape[0], fixed_radii_cm.size), np.nan)
151     fixed_m = fixed_radii_cm * 0.01
152     for i, (row, radius_m) in enumerate(zip(moisture_rows, radii_m)):
153         valid = fixed_m < radius_m - 1.0e-13
154         output[i, valid] = np.interp(fixed_m[valid] / radius_m, xi, row)
155     return output
156
157
158 def solve_until_dry(
159     environment: EnvironmentData,
160     radius_data: RadiusData,
161     parameters: PhysicalParameters,
162     config: NumericalConfig,
163     progress_label: str | None = None,
164     integration_method: str = "BDF",

```

```

165 ) -> SimulationResult:
166     if integration_method not in {"BDF", "Radau"}:
167         raise ValueError("integration_method must be 'BDF' or 'Radau'")
168     config.validate()
169     xi = make_reference_grid(
170         config.radial_cells,
171         config.reference_surface_layer,
172         config.surface_refinement_factor,
173     )
174     system = MovingBoundarySystem(environment, radius_data, parameters, xi)
175     node_count = xi.size
176     # The sparsity graph depends only on the fixed reference grid. Construct
177     # it once; rebuilding the same matrix in every six-hour integration
178     # segment is pure overhead and changes no solver semantics.
179     jacobian_sparsity = make_jacobian_sparsity(node_count)
180     state = np.concatenate(
181         (
182             np.full(node_count, parameters.initial_temperature_c),
183             np.full(node_count, parameters.initial_moisture_kg_kg),
184             np.asarray([0.0]),
185         )
186     )
187     absolute_tolerance = np.concatenate(
188         (
189             np.full(node_count, config.temperature_absolute_tolerance),
190             np.full(node_count, config.moisture_absolute_tolerance),
191             np.asarray([config.moisture_absolute_tolerance]),
192         )
193     )
194     xi_samples = np.linspace(0.0, 1.0, 101)
195     physical_radii_cm = np.arange(20, dtype=float) * 0.1
196     time_chunks: list[np.ndarray] = []
197     temperature_chunks: list[np.ndarray] = []
198     moisture_chunks: list[np.ndarray] = []
199     physical_chunks: list[np.ndarray] = []
200     surface_chunks: list[np.ndarray] = []
201     maximum_chunks: list[np.ndarray] = []
202     radius_chunks: list[np.ndarray] = []
203     inventory_chunks: list[np.ndarray] = []
204     boundary_integral_chunks: list[np.ndarray] = []
205
206     minima = np.full(6, np.inf)
207     maxima = np.full(6, -np.inf)
208     nfev = njev = nlu = segments = 0
209     maximum_at_center = True
210     crossing_s: float | None = None
211     started = time.perf_counter()
212     current_time = 0.0
213
214     while current_time < config.maximum_duration_s and crossing_s is None:
215         if current_time < config.environment_data_end_s:
216             segment_end = min(config.environment_data_end_s, config.maximum_duration_s)
217             maximum_step = config.initial_maximum_step_s
218         else:
219             segment_end = min(
220                 current_time + config.drying_segment_s,
221                 config.maximum_duration_s,
222             )
223             maximum_step = config.drying_maximum_step_s
224         eval_times = np.arange(
225             current_time,
226             segment_end + 0.5 * config.output_interval_s,
227             config.output_interval_s,
228         )

```

```

229     if progress_label:
230         print(
231             f"[{progress_label}] {current_time / 3600:.1f}--"
232             f"{segment_end / 3600:.1f} h, max_step={maximum_step:g} s",
233             flush=True,
234         )
235     # The last state is a passive flux integral. Its Jacobian column is
236     # identically zero, so SciPy's finite-difference step controller can
237     # harmlessly overflow its unused perturbation factor. Suppress only
238     # those two narrowly scoped internal warnings.
239     with warnings.catch_warnings():
240         warnings.filterwarnings(
241             "ignore",
242             message="overflow encountered in multiply",
243             category=RuntimeWarning,
244             module=r"scipy\.integrate\._ivp\.common",
245         )
246         warnings.filterwarnings(
247             "ignore",
248             message="invalid value encountered in multiply",
249             category=RuntimeWarning,
250             module=r"scipy\.integrate\._ivp\.common",
251         )
252         solution = solve_ivp(
253             system,
254             (current_time, segment_end),
255             state,
256             method=integration_method,
257             t_eval=eval_times,
258             dense_output=True,
259             rtol=config.relative_tolerance,
260             atol=absolute_tolerance,
261             jac_sparsity=jacobian_sparsity,
262             max_step=maximum_step,
263             first_step=min(config.first_step_s, segment_end - current_time),
264         )
265     if not solution.success:
266         raise RuntimeError(
267             f"[integration_method] integration failed: {solution.message}"
268         )
269     if not np.all(np.isfinite(solution.y)):
270         raise RuntimeError(
271             f"[integration_method] solution contains non-finite values"
272         )
273     temperatures_nodes = solution.y[:node_count].T
274     moistures_nodes = solution.y[node_count : 2 * node_count].T
275     boundary_integral = solution.y[2 * node_count]
276     if np.min(moistures_nodes) <= 0.0:
277         raise RuntimeError("computed non-positive moisture")
278
279     densities, heat_capacities, conductivities, diffusivities = (
280         parameters.evaluate_properties(
281             moistures_nodes.ravel(), temperatures_nodes.ravel()
282         )
283     )
284     tracked = (
285         temperatures_nodes,
286         moistures_nodes,
287         densities,
288         heat_capacities,
289         conductivities,
290         diffusivities,
291     )
292     for j, values in enumerate(tracked):

```



```

293         minima[j] = min(minima[j], float(np.min(values)))
294         maxima[j] = max(maxima[j], float(np.max(values)))
295
296     local_maximum = np.max(moistures_nodes, axis=1)
297     maximum_at_center = maximum_at_center and bool(
298         np.all(local_maximum - moistures_nodes[:, 0] <= 1.0e-10)
299     )
300     crossing_candidates = np.flatnonzero(
301         (local_maximum[:-1] > parameters.drying_threshold_kg_kg)
302         & (local_maximum[1:] <= parameters.drying_threshold_kg_kg)
303     )
304     if crossing_candidates.size:
305         j = int(crossing_candidates[0])
306
307         def event_value(t_s: float) -> float:
308             dense_state = solution.sol(t_s)
309             return (
310                 float(np.max(dense_state[node_count : 2 * node_count]))
311                 - parameters.drying_threshold_kg_kg
312             )
313
314         crossing_s = float(
315             brentq(
316                 event_value,
317                 float(solution.t[j]),
318                 float(solution.t[j + 1]),
319                 xtol=1.0e-7,
320                 rtol=1.0e-13,
321             )
322         )
323
324     radii = radius_data.interpolate(solution.t)
325     sampled_temperature = interpolate_rows(xi, temperatures_nodes, xi_samples)
326     sampled_moisture = interpolate_rows(xi, moistures_nodes, xi_samples)
327     physical_moisture = extract_physical_moistures(
328         xi, moistures_nodes, radii, physical_radii_cm
329     )
330     inventory = moistures_nodes @ system.weights
331     start_index = 0 if segments == 0 else 1
332     sl = slice(start_index, None)
333     time_chunks.append(solution.t[sl].copy())
334     temperature_chunks.append(sampled_temperature[sl].copy())
335     moisture_chunks.append(sampled_moisture[sl].copy())
336     physical_chunks.append(physical_moisture[sl].copy())
337     surface_chunks.append(moistures_nodes[sl, -1].copy())
338     maximum_chunks.append(local_maximum[sl].copy())
339     radius_chunks.append(radii[sl].copy())
340     inventory_chunks.append(inventory[sl].copy())
341     boundary_integral_chunks.append(boundary_integral[sl].copy())
342
343     nfev += int(solution.nfev)
344     njev += int(solution.njev)
345     nlu += int(solution.nlu)
346     segments += 1
347     state = solution.y[:, -1].copy()
348     current_time = segment_end
349
350     runtime_s = time.perf_counter() - started
351     if crossing_s is None:
352         raise RuntimeError(
353             f"drying threshold was not reached within {config.maximum_duration_s / 3600:g} h"
354         )
355
356     times = np.concatenate(time_chunks)

```

```

357 temperatures = np.vstack(temperature_chunks)
358 moistures = np.vstack(moisture_chunks)
359 physical_moistures = np.vstack(physical_chunks)
360 surface_moistures = np.concatenate(surface_chunks)
361 maximum_moistures = np.concatenate(maximum_chunks)
362 radii = np.concatenate(radius_chunks)
363 inventories = np.concatenate(inventory_chunks)
364 boundary_integrals = np.concatenate(boundary_integral_chunks)
365 strict_indices = np.flatnonzero(
366     (times > 0.0)
367     & (maximum_moistures < parameters.drying_threshold_kg_kg)
368 )
369 if strict_indices.size == 0:
370     raise RuntimeError("post-crossing segment did not contain a strict output time")
371 first_strict_output_s = int(round(float(times[int(strict_indices[0])])))
372
373 residual = inventories - inventories[0] - boundary_integrals
374 residual_scale = np.maximum.reduce(
375     (
376         np.abs(inventories - inventories[0]),
377         np.abs(boundary_integrals),
378         np.full_like(residual, 1.0e-30),
379     )
380 )
381
382 diagnostics = SolverDiagnostics(
383     runtime_s=runtime_s,
384     function_evaluations=nfev,
385     jacobian_evaluations=njev,
386     lu_decompositions=nlu,
387     integration_segments=segments,
388     returned_time_points=int(times.size),
389     min_temperature_c=minima[0],
390     max_temperature_c=maxima[0],
391     min_moisture_kg_kg=minima[1],
392     max_moisture_kg_kg=maxima[1],
393     min_density_kg_m3=minima[2],
394     max_density_kg_m3=maxima[2],
395     min_heat_capacity_j_kg_k=minima[3],
396     max_heat_capacity_j_kg_k=maxima[3],
397     min_conductivity_w_m_k=minima[4],
398     max_conductivity_w_m_k=maxima[4],
399     min_diffusivity_m2_s=minima[5],
400     max_diffusivity_m2_s=maxima[5],
401     max_output_interval_balance_abs=float(np.max(np.abs(residual))),
402     max_output_interval_balance_rel=float(
403         np.max(np.abs(residual[1:]) / residual_scale[1:])
404     ),
405     maximum_location_was_center=maximum_at_center,
406 )
407 if progress_label:
408     print(
409         f"[{progress_label}] crossing={crossing_s / 3600:.6f} h, "
410         f"runtime={runtime_s:.2f} s, nfev={nfev}, nlu={nlu}",
411         flush=True,
412     )
413 return SimulationResult(
414     times_s=np rint(times).astype(int),
415     xi_samples=xi_samples,
416     temperatures_c=temperatures,
417     moistures_kg_kg=moistures,
418     physical_radii_cm=physical_radii_cm,
419     physical_moistures_kg_kg=physical_moistures,
420     surface_moistures_kg_kg=surface_moistures,

```

```

421         maximum_moistures_kg_kg=maximum_moistures,
422         radii_m=radii,
423         drying_crossing_s=crossing_s,
424         first_strict_output_s=first_strict_output_s,
425         diagnostics=diagnostics,
426         config=config,
427     )
428
429
430 def common_field_comparison(a: SimulationResult, b: SimulationResult) -> dict:
431     common_end = min(int(a.times_s[-1]), int(b.times_s[-1]))
432     count = common_end // int(a.config.output_interval_s) + 1
433     if not np.array_equal(a.times_s[:count], b.times_s[:count]):
434         raise ValueError("comparison time grids do not align")
435     difference = np.abs(a.moistures_kg_kg[:count] - b.moistures_kg_kg[:count])
436     flat_index = int(np.argmax(difference))
437     time_index, space_index = np.unravel_index(flat_index, difference.shape)
438     return {
439         "max_abs_difference": float(difference[time_index, space_index]),
440         "rms_difference": float(np.sqrt(np.mean(difference * difference))),
441         "max_location_time_s": int(a.times_s[time_index]),
442         "max_location_xi": float(a.xi_samples[space_index]),
443         "common_end_time_s": common_end,
444         "drying_crossing_difference_s": abs(a.drying_crossing_s - b.drying_crossing_s),
445     }
446
447
448 def spatial_convergence(
449     coarse: SimulationResult,
450     medium: SimulationResult,
451     fine: SimulationResult,
452 ) -> dict:
453     cm = common_field_comparison(coarse, medium)
454     mf = common_field_comparison(medium, fine)
455     if cm["max_abs_difference"] > 0.0 and mf["max_abs_difference"] > 0.0:
456         order = math.log(cm["max_abs_difference"] / mf["max_abs_difference"], 2.)
457         if order > 0.0:
458             estimate = mf["max_abs_difference"] / (2.0**order - 1.0)
459         else:
460             estimate = math.inf
461     else:
462         order = math.nan
463         estimate = math.nan
464     return {
465         "coarse_to_medium": cm,
466         "medium_to_fine": mf,
467         "observed_order": order,
468         "estimated_fine_grid_max_error": estimate,
469         "estimated_fine_grid_max_error_with_1p25_safety": 1.25 * estimate,
470         "crossing_times_s": {
471             "coarse": coarse.drying_crossing_s,
472             "medium": medium.drying_crossing_s,
473             "fine": fine.drying_crossing_s,
474         },
475     }
476
477
478 def choose_safe_end_time(
479     fine: SimulationResult,
480     numerical_error_bound: float,
481     threshold: float,
482 ) -> int:
483     # The extra half unit in the fourth decimal place ensures the submitted
484     # table visibly rounds below 0.1500 as well as satisfying the full-precision

```

```

485     # physical threshold after the numerical error allowance is added.
486     displayed_threshold = threshold - 0.5e-4
487     candidates = np.flatnonzero(
488         (fine.times_s > 0)
489         & (
490             fine.maximum_moistures_kg_kg + numerical_error_bound
491             < displayed_threshold
492         )
493     )
494     if candidates.size == 0:
495         raise RuntimeError("stored post-crossing interval is too short for safety margin")
496     return int(fine.times_s[int(candidates[0])])
497

```